

# NL2SQL: PRODUCTION CONVERSATIONAL AI NATURAL LANGUAGE TO SQL SYSTEM

Comprehensive Technical Project Analysis, Software Architecture Audit & Technical Jury Defense Guide

|                     |                                                              |                       |                                                     |
|---------------------|--------------------------------------------------------------|-----------------------|-----------------------------------------------------|
| Target Audience:    | Software Engineering, AI/ML & System Architecture Jury Panel | Architecture Pattern: | Decoupled SPA + REST API + Universal DB Abstraction |
| Core AI Engine:     | Google Gemini (Flash-Lite) + SentenceTransformer + FAISS     | Validation Engine:    | sqlglot AST Engine + Staged Write State Machine     |
| Multilingual Scope: | English, Tamil Script (Unicode), Thanglish (Phonetic)        | Verification Status:  | 166 Automated Test Cases Passed (100% Pass Rate)    |

## Document Organization & Table of Contents

|                                            |                                             |                                         |
|--------------------------------------------|---------------------------------------------|-----------------------------------------|
| Sec 1: Project Overview & System Topology  | Sec 11: Comprehensive Security Architecture | Sec 21: Verification & Testing Matrix   |
| Sec 2: Complete Technology Stack Audit     | Sec 12: Error Handling & Fault Tolerance    | Sec 22: Deployment & Containerization   |
| Sec 3: Project Directory Hierarchy         | Sec 13: Latency & Performance Profile       | Sec 23: Technical Weaknesses & Risks    |
| Sec 4: File & Module Inventory Sheet       | Sec 14: System Scalability Roadmap          | Sec 24: 5-Minute Technical Presentation |
| Sec 5: Deep Source Code Inspection         | Sec 15: Architectural Design Trade-Offs     | Sec 25: 30-Second Technical Pitch       |
| Sec 6: Critical Code Block Analysis        | Sec 16: Module Dependency Topology          | Sec 26: 2-Minute Architectural Briefing |
| Sec 7: Complete End-to-End Data Flows      | Sec 17: Jury Must-Know Core Concepts        | Sec 27: Rapid 10-Minute Revision Sheet  |
| Sec 8: RESTful API Endpoint Audit          | Sec 18: Technical Jury Question Bank (A-R)  | Sec 28: Exhaustive Technical Glossary   |
| Sec 9: Relational Schema & Storage Design  | Sec 19: Adversarial Jury Cross-Questions    |                                         |
| Sec 10: AI, LLM, RAG & Multilingual Engine | Sec 20: 'Show Me The Code' Walkthroughs     |                                         |

## SECTION 1 — PROJECT OVERVIEW

### 1.1 Project Identity & Executive Summary

**NL2SQL** is an enterprise-grade, conversational Natural Language-to-SQL (NL2SQL) system engineered to empower non-technical domain users—such as hospital administrators, retail inventory managers, business analysts, and executives—to interrogate and safely manipulate relational databases without writing SQL syntax. Users interact via text or voice in **English**, **Tamil script**, or **Thanglish** (Tamil phonetic transliteration in Latin script). The system returns syntactically verified SQL, natural language explanations, automated Chart.js visualizations, and an educational 'Learn SQL' interactive breakdown.

### 1.2 The Problem Being Solved

Enterprise relational databases remain siloed behind the technical barrier of SQL. While non-technical staff require daily data insights, conventional approaches suffer from severe operational friction:

- **Developer Bottleneck:** Business analysts and operators must request ad-hoc SQL reports from engineering teams, introducing delays of hours or days.
- **Hallucination & Syntax Failures in Naive LLMs:** Generic LLMs frequently hallucinate non-existent table and column names, omit necessary JOIN conditions, or produce dialect-incompatible syntax.
- **Catastrophic Data Loss & Security Risks:** Connecting LLMs directly to databases exposes systems to SQL injection, accidental DROP/TRUNCATE execution, and unconstrained UPDATE/DELETE mutations without WHERE clauses.
- **Linguistic Exclusivity:** Most NL2SQL systems operate exclusively in English, isolating regional domain workers in diverse linguistic settings such as Tamil Nadu, India.

1.3 Target Users

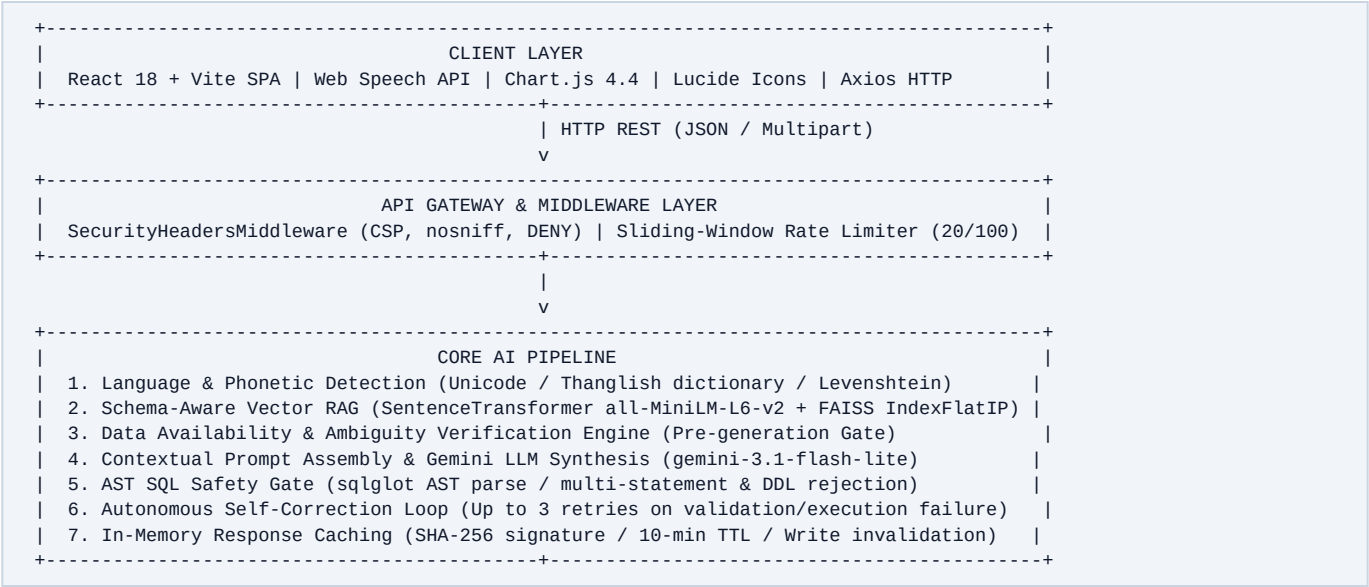
- **Clinical & Operational Hospital Staff:** Doctors and administrators querying patient admissions, doctor assignments, and billing without technical assistance.
- **E-Commerce & Retail Merchants:** Store operators monitoring customer orders, inventory stock levels, category sales, and product performance.
- **Business & Financial Analysts:** Professionals requiring instant statistical summaries, KPI cards, and trend visualizations from connected SQLite or PostgreSQL databases.
- **Engineering Teams & Students:** Developers and learners utilizing the pedagogical 'Learn SQL' engine to understand relational query construction block-by-block.

1.4 Main Objectives & Key Architectural Features

The system satisfies four primary technical objectives: **Correctness**, **Safety**, **Accessibility**, and **Observability**. Key capabilities implemented in the codebase include:

- **Universal Database Support:** An object-oriented adapter architecture (BaseDatabaseAdapter) managing live SQLite files, PostgreSQL network endpoints, and uploaded CSV/SQL files.
- **Deep Schema & Constraint Introspection:** Automated extraction of table structures, primary keys, foreign keys, nullability, row counts, and distinct non-leaking sample values.
- **Schema-Aware Vector RAG (FAISS + all-MiniLM-L6-v2):** Vector similarity retrieval dynamically trimming large database schemas (> 4 tables) to the top relevant tables, avoiding LLM prompt bloat while preserving small schemas intact.
- **Trilingual NLP Engine:** Deterministic language detection and prompt grounding for English, native Tamil script (Unicode block \u0B80-\u0BFF), and Thanglish.
- **Phonetic Speech Normalization:** Browser-native Web Speech API capture paired with backend phonetic mishearing correction (e.g. 'pashents' → 'patients').
- **Ambiguity & Boundary Pre-Checks:** Pre-generation interception for unconstrained ranking terms ('top patients') and out-of-schema entities (asking for flights on a hospital DB).
- **Hardened AST Validation (sqlglot):** Strict Abstract Syntax Tree parsing rejecting multi-statement injection, comments, destructive DDL (DROP, ALTER, TRUNCATE), and unconstrained writes.
- **Guarded Two-Phase Write Confirmation:** Strict READ-ONLY default; write operations (INSERT, UPDATE, DELETE) are staged in memory (\_PENDING\_WRITES) and executed only upon explicit user confirmation.
- **Autonomous Self-Correction Loop:** Automated retry mechanism (up to 3 attempts) passing validation or database execution syntax errors back to Gemini for real-time query repair.
- **In-Memory Deterministic Caching:** SHA-256 hashed response cache with 10-minute TTL, automatically invalidated upon confirmed write modifications.
- **Zero-Result Reasoning Engine:** Evaluates empty dataset results against schema sample values to explain *why* zero rows were returned.

1.5 High-Level Architectural Diagram



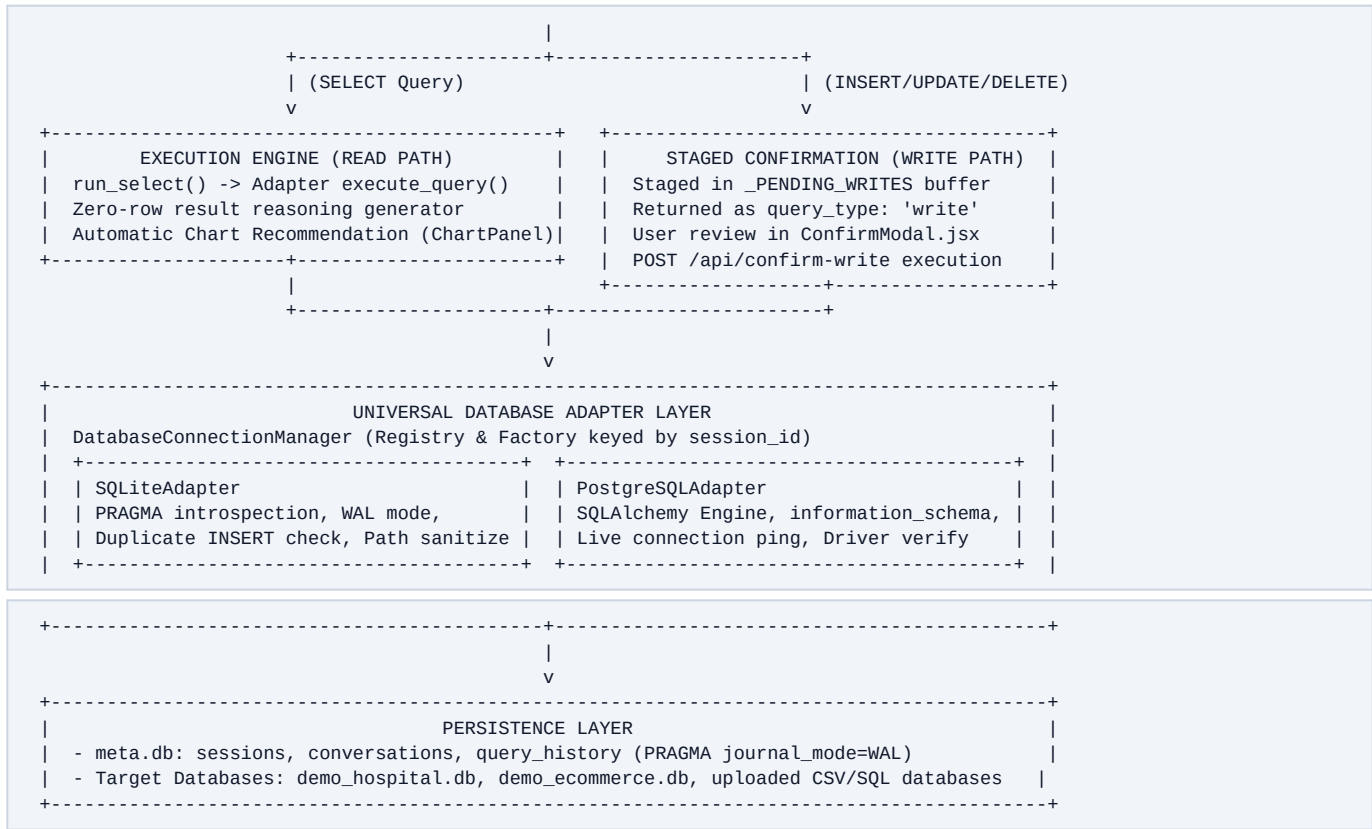


Figure: End-to-End System Topology & Interaction Flow

## SECTION 2 — COMPLETE TECHNOLOGY STACK

Every technology in the codebase was deliberately selected to fulfill specific architectural guarantees: type safety, low latency, robust AST manipulation, mathematical semantic search, and defensive security.

| Layer          | Technology                 | Version        | Exact Purpose in Project                     | Architectural Rationale                                                          |
|----------------|----------------------------|----------------|----------------------------------------------|----------------------------------------------------------------------------------|
| Frontend UI    | React.js                   | 18.3.1         | SPA component hierarchy and state management | Declarative rendering, reactive state for chat streams and modals.               |
| Frontend Build | Vite                       | 5.4.2          | Development server and asset bundler         | Near-instant HMR, ES module bundling, content-hashed asset production.           |
| Styling        | Tailwind CSS               | 3.4.1          | Utility-first responsive design system       | Eliminates CSS bloat via PurgeCSS; consistent slate/teal dark/light palette.     |
| Icons          | Lucide React               | 0.344.0        | UI vector iconography                        | Tree-shakable SVG icon set with zero runtime overhead.                           |
| Visualization  | Chart.js / react-chartjs-2 | 4.4.8 / 5.3.0  | Dynamic data chart rendering                 | HTML5 Canvas rendering of bar charts, line graphs, and KPI cards.                |
| Speech Input   | Web Speech API             | Browser Native | Speech-to-text audio capture                 | Zero-latency in-browser transcription without costly cloud audio API calls.      |
| HTTP Client    | Axios                      | 1.7.9          | REST communication with FastAPI backend      | Interceptors, unified error normalization, configurable base URLs.               |
| Backend Web    | FastAPI                    | 0.139.0        | Asynchronous ASGI API gateway                | High-throughput ASGI server, automatic OpenAPI docs, Pydantic schema validation. |
| Server Engine  | Uvicorn                    | 0.51.0         | ASGI HTTP server implementation              | Production ASGI worker handling HTTP/1.1 and WebSockets efficiently.             |

| Layer            | Technology              | Version       | Exact Purpose in Project              | Architectural Rationale                                                     |
|------------------|-------------------------|---------------|---------------------------------------|-----------------------------------------------------------------------------|
| Validation       | Pydantic                | 2.13.4        | Data parsing and contract enforcement | Type validation, serialization, and automatic schema documentation.         |
| Middleware       | Starlette               | 1.3.1         | HTTP kernel, security headers, CORS   | Lightweight HTTP primitives, CORS middleware, base middleware dispatch.     |
| Primary AI LLM   | Google Gemini API       | google-gen-ai | NL to SQL synthesis & explanation     | Large context window, high reasoning speed, native JSON schema mode.        |
| Embeddings       | SentenceTransformers    | 3.0+          | Semantic dense vector generation      | Local 384-dimensional vector embeddings via all-MiniLM-L6-v2.               |
| Vector Search    | FAISS                   | faiss-cpu     | Schema-aware similarity retrieval     | Millisecond cosine similarity search over table schema representations.     |
| SQL Parsing      | sqlglot                 | 25.0+         | AST SQL validation & comment removal  | Dialect-aware AST construction, statement isolation, syntax checking.       |
| Relational DB    | SQLite3                 | 3.45+         | Metadata store & demo databases       | Zero-configuration serverless ACID relational storage with WAL mode.        |
| Enterprise DB    | PostgreSQL / SQLAlchemy | 2.0.51        | Enterprise database adapter & ORM     | Universal connection interface, information_schema introspection.           |
| Containerization | Docker                  | Multi-stage   | Unified build and runtime deployment  | Two-stage build: Node 20 asset bundling + Python 3.11 slim runtime.         |
| Testing Engine   | pytest / pytest-cov     | 9.1.1 / 7.1.0 | Automated test suite & coverage       | Parametric test suites, Starlette TestClient integration, coverage metrics. |

2.1 In-Depth Analysis of Critical Technologies

1. Google Gemini API (gemini-3.1-flash-lite)

**What it is:** Google's state-of-the-art lightweight generative language model.

**Where used:** app/services/sql\_generator.py in generate\_sql() and regenerate\_sql().

**Why used:** Offers sub-second inference latency, robust multilingual understanding (native Tamil script and Thanglish), and strict adherence to JSON output schemas.

**What happens if removed:** Natural language query synthesis halts. The system would require replacing it with an alternative LLM client (e.g. OpenAI GPT-4o or a local Ollama model).

**Alternatives:** OpenAI GPT-4o-mini, Anthropic Claude 3.5 Haiku, local CodeLlama 13B via vLLM.

**Key Concepts:** System instructions, temperature (set to 0.1 for deterministic SQL), JSON mode, few-shot prompt grounding.

2. sqlglot (AST Parsing & SQL Transpilation Engine)

**What it is:** A pure-Python SQL parser, transpiler, and Abstract Syntax Tree (AST) analyzer.

**Where used:** app/services/sql\_validator.py and app/services/execution\_engine.py.

**Why used:** Regular expressions are fundamentally incapable of validating recursive SQL grammars. sqlglot parses statements into mathematical AST trees, identifying individual expressions (exp.Select, exp.Insert, exp.Update, exp.Drop), detecting stacked queries separated by semicolons, and validating required clauses like WHERE.

**What happens if removed:** The system would lose its primary security defense, exposing the database to SQL injection bypasses, stacked destructive commands, and unconstrained table updates.

**Alternatives:** sqlparse (token-based only, cannot build semantic AST), pglast (PostgreSQL only, C dependency).

**Key Concepts:** Abstract Syntax Tree, expression traversal (stmt.find(exp.Where)), dialect normalization.

3. SentenceTransformers ('all-MiniLM-L6-v2') & FAISS

**What it is:** A pre-trained bi-encoder neural network mapping text to 384-dimensional dense vectors, paired with Facebook AI Similarity Search.

**Where used:** app/services/rag\_service.py.

**Why used:** When a database contains dozens or hundreds of tables, feeding the entire schema into an LLM prompt exhausts context limits, degrades reasoning accuracy, and skyrockets latency. FAISS indexes table metadata summaries and retrieves only the top-4 relevant tables.

**What happens if removed:** For large schemas, prompt token count explodes, increasing API cost and latency by up to 400%, and confusing the LLM.

**Alternatives:** ChromaDB, Qdrant, BM25 keyword search (lacks semantic synonym matching).

**Key Concepts:** Dense vector embeddings, cosine similarity via IndexFlatIP, normalized vector dot product.

#### 4. SQLite with Write-Ahead Logging (WAL)

**What it is:** An in-process, zero-configuration relational database engine configured in WAL mode.

**Where used:** meta.db (sessions, conversations, history) and demo databases (demo\_hospital.db, demo\_ecommerce.db).

**Why used:** Eliminates separate database server overhead. Standard SQLite locks the entire database file during writes; enabling PRAGMA journal\_mode=WAL and PRAGMA synchronous=NORMAL permits concurrent readers while a write proceeds.

**What happens if removed:** Metadata persistence fails, multi-turn conversation history is lost across restarts, and demo datasets cannot be queried.

**Alternatives:** PostgreSQL, MySQL, DuckDB.

**Key Concepts:** Write-Ahead Logging, ACID transactions, check\_same\_thread=False, PRAGMA tuning.

## SECTION 3 — COMPLETE PROJECT FOLDER STRUCTURE

The project is structured as a decoupled monorepo containing a modern React frontend, a FastAPI Python backend, persisted relational datasets, automated test suites, deployment scripts, and architecture documentation.

```

NL2SQL-Full-project-main/
|-- Dockerfile           # Multi-stage Dockerfile: Node 20 build + Python 3.11 slim runtime
|-- API.md               # Complete REST API specification and endpoint documentation
|-- ARCHITECTURE.md      # System architecture, layer contracts, and security rules
|-- IMPLEMENTATION_REPORT.md # Implementation milestone verification report
|-- LIMITATIONS.md       # Operational boundaries, design trade-offs, and roadmap
|-- README.md            # High-level overview, quickstart instructions, and test guide
|-- TEST_REPORT.md       # Test suite verification breakdown (166 automated tests)
|
+-- frontend/            # React 18 + Vite Single Page Application (SPA)
|   |-- package.json      # Frontend dependencies (React, Vite, Chart.js, Tailwind, Axios)
|   |-- vite.config.js    # Vite bundler configuration with proxy and asset hashing
|   |-- tailwind.config.js # Tailwind utility styling theme and color palette definitions
|   |-- index.html        # Single page HTML entry point with viewport and metadata
|   +-- src/
|       |-- App.jsx       # Root application coordinator managing session, theme, and screens
|       |-- App.css / index.css # Custom styling rules and Tailwind utility directives
|       |-- main.jsx      # React DOM mounting entry point
|       +-- api/
|           |-- client.js  # Axios REST client with error normalization and endpoint wrappers
|           +-- components/
|               |-- ConnectDBScreen.jsx # Initial landing screen: demo selector, CSV/SQL upload, URI input
|               |
|               |-- ChatWindow.jsx      # Main chat workspace: message stream, query input, voice button
|               |-- MessageBubble.jsx   # Interactive message card: SQL drawer, chart viewer, feedback
|               |-- ChartPanel.jsx      # Dynamic Chart.js renderer (Bar, Line, KPI, Data Table)
|               |-- ConfirmModal.jsx    # Modal dialog for two-phase write confirmation (INSERT/UPDATE/DELETE)
|               |-- LearnSQLModal.jsx   # Interactive pedagogical popup explaining generated SQL
|               |-- SQLDrawer.jsx       # Slide-out syntax-highlighted SQL drawer
|               |-- SQLPreviewPanel.jsx # Live SQL preview pane with execution controls
|               |-- VoiceButton.jsx     # Web Speech API speech-to-text button with recording state
|               |-- HistorySidebar.jsx  # Sidebar listing historical conversation threads with search
|               +-- HelpSidebar.jsx     # Contextual help pane showing schema hints and sample queries
|           +-- utils/
|               |-- learnSqlGenerator.js # In-browser pedagogical SQL engine generating 8-stage lessons
|               |-- messageFormatter.js  # Sanitizer and markdown formatter for chat text
|               +-- media.js             # Audio playback helpers for voice interaction
|           |
|       +-- nl2sql-backend/             # Python 3.11 FastAPI Application Engine
|           |-- requirements.txt         # Python package requirements (FastAPI, PyTorch, FAISS, sqlglot)
|           |-- pytest.ini               # Pytest configuration (test discovery, warning filters)
|           |-- run_tests.py             # Standalone test runner script
|           +-- app/
|               |-- main.py              # FastAPI app initialization, CORS, security middleware, SPA mount
|               +-- config.py            # Environment variables, directory paths, Gemini API key hygiene

```

```
|      +-- database/                # Universal Database Abstraction Layer
|      |  |-- base.py                # BaseDatabaseAdapter abstract class defining universal contracts
|      |  |-- manager.py             # DatabaseConnectionManager factory and adapter registry
|      |  |-- sqlite_adapter.py      # SQLiteAdapter implementation (demo DBs, uploaded files, WAL)
|      |  +-- postgres_adapter.py    # PostgreSQLAdapter implementation (SQLAlchemy, information_schema)
|      +-- models/
|      |  +-- meta_db.py             # SQLAlchemy models for meta.db: sessions, conversations, query_history
|      +-- routers/
|      |  |-- connect_db.py          # Endpoints: /api/connect-db, /api/upload-data, /api/session-status
|      |  |-- query.py               # Endpoints: /api/query, /api/confirm-write, rate limiting
|      |  +-- conversations.py       # Endpoints: /api/conversations, /messages, title generation
|      +-- services/
|      |  |-- sql_generator.py        # Core AI engine: Gemini synthesis, multilingual, phonetic, ambiguity
|      |  |-- sql_validator.py        # Security safety gate: sqlglot AST parsing, injection/DDL block
|      |  |-- execution_engine.py     # Safe SELECT execution, transactional writes, duplicate check
|      |  |-- rag_service.py          # Schema-aware RAG: SentenceTransformer embeddings + FAISS index
|      |  +-- session_store.py        # In-memory session registry with fallback restoration from meta.db
|      +-- data/
|      |  |-- meta.db                # System metadata store (sessions, conversations, query history)
|      |  |-- demo_hospital.db        # Hospital demo database (patients, doctors, appointments, bills)
|      |  +-- demo_ecommerce.db       # E-commerce demo database (customers, products, orders, items)
|      +-- tests/
|      |  |-- conftest.py             # Pytest fixtures: test database setup, sample sessions, TestClient
|      |  |-- test_adapters_and_writes.py # Tests for adapter interface, schema extraction, writes
|      |  |-- test_sql_validator.py    # 44 tests for AST validation, comments, DDL rejection, WHERE clauses
|      |  |-- test_security.py         # 23 tests for rate limiting, security headers, file upload guards
|      |  |-- test_rag.py              # 11 tests for FAISS index lifecycle and table retrieval
|      |  |-- test_multilingual.py     # 9 tests for Tamil script, Thanglish, and translation integrity
|      |  |-- test_self_correction.py  # 8 tests for autonomous retry loop and error feedback
|      +-- ...                        # Additional unit and integration test suites
```

Figure: Complete Project Directory Hierarchy

## SECTION 4 — COMPLETE FILE & MODULE INVENTORY

The following master inventory specifies the exact architectural layer, responsibility, critical functions, and dependencies for every production source file in the repository.

| File                             | Module / Layer | Purpose                                                            | Important Classes / Functions                                 | Dependencies                           |
|----------------------------------|----------------|--------------------------------------------------------------------|---------------------------------------------------------------|----------------------------------------|
| app/main.py                      | API Gateway    | ASGI application initialization, security middleware, SPA routing  | SecurityHeadersMiddleware, lifespan(), serve_spa_frontend()   | FastAPI, Starlette, config, routers    |
| app/config.py                    | Configuration  | Path resolution, environment configuration, Gemini API key hygiene | BASE_DIR, DATA_DIR, IS_PRODUCTION, GEMINI_API_KEY             | dotenv, os, pathlib                    |
| app/database/base.py             | DB Abstraction | Abstract base class defining universal database adapter interface  | BaseDatabaseAdapter, extract_full_schema(), execute_query()   | abc, typing                            |
| app/database/manager.py          | DB Abstraction | Factory and registry managing active adapters keyed by session_id  | DatabaseConnectionManager, create_adapter(), get_adapter()    | base, sqlite_adapter, postgres_adapter |
| app/database/sqlite_adapter.py   | DB Abstraction | Concrete SQLite adapter with deep schema extraction and WAL tuning | SQLiteAdapter, extract_full_schema(), execute_write()         | sqlite3, base, execution_engine        |
| app/database/postgres_adapter.py | DB Abstraction | Concrete PostgreSQL adapter inspecting information_schema          | PostgreSQLAdapter, validate_connection(), _check_driver()     | SQLAlchemy, psycopg2/asyncpg           |
| app/models/meta_db.py            | Persistence    | SQLAlchemy models and database initialization for system metadata  | SessionModel, ConversationModel, QueryHistoryModel, init_db() | SQLAlchemy, sqlite3, config            |
| app/routers/connect_db.py        | REST API       | Database connection, CSV/SQL upload parsing, and schema inspection | connect_db(), upload_data(), inspect_db_schema_and_samples()  | FastAPI, SQLAlchemy, manager, config   |

| File                                      | Module / Layer    | Purpose                                                               | Important Classes / Functions                                      | Dependencies                         |
|-------------------------------------------|-------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------|
| app/routers/query.py                      | REST API          | Query translation, two-tier rate limiting, staged writes, retry loop  | handle_query(), confirm_write(), check_rate_limit()                | sql_generator, sql_validator, engine |
| app/routers/conversations.py              | REST API          | Conversation thread management, history retrieval, auto-titling       | create_conversation(), list_conversations(), get_messages()        | meta_db, FastAPI, pydantic           |
| app/services/sql_generator.py             | Core AI Engine    | Gemini synthesis, multilingual detection, phonetic correction, cache  | generate_sql(), regenerate_sql(), detect_input_language()          | google.generativeai, rag_service     |
| app/services/sql_validator.py             | Security / Safety | sqlglot AST validation, comment stripping, DDL rejection              | validate_sql(), validate_write_sql(), _strip_strings()             | sqlglot, exp, re                     |
| app/services/execution_engine.py          | Execution         | Safe SELECT execution, transactional writes, duplicate insert check   | run_select(), run_write(), check_duplicate_insert()                | sqlite3, sqlglot, manager            |
| app/services/rag_service.py               | AI / Retrieval    | SentenceTransformers + FAISS schema-aware table retrieval             | build_schema_index(), retrieve_relevant_tables()                   | sentence_transformers, faiss, numpy  |
| app/services/session_store.py             | State Store       | In-memory session registry with fallback restoration from meta.db     | get_session(), set_session(), SESSION_STORE                        | meta_db, rag_service                 |
| frontend/src/App.jsx                      | Frontend Root     | Top-level application orchestrator, theme manager, session gate       | App(), handleConnect(), handleDisconnect(), refreshConversations() | React, ConnectDBScreen, ChatWindow   |
| frontend/src/api/client.js                | Frontend API      | Axios client with base URL handling and unified error parsing         | connectDB(), sendQuery(), confirmWrite(), getSessionStatus()       | axios                                |
| frontend/src/components/ChatWindow.jsx    | Frontend UI       | Main chat interface: message stream, query dispatch, follow-up state  | ChatWindow(), handleSend(), handleVoiceTranscript()                | MessageBubble, VoiceButton, client   |
| frontend/src/components/MessageBubble.jsx | Frontend UI       | Message rendering: SQL drawer, ChartPanel, LearnSQLModal trigger      | MessageBubble(), renderChart(), renderSqlPreview()                 | ChartPanel, LearnSQLModal, Lucide    |
| frontend/src/components/ConfirmModal.jsx  | Frontend UI       | Modal dialog for staged two-phase write confirmation                  | ConfirmModal(), handleConfirm(), handleCancel()                    | React, Lucide React                  |
| frontend/src/components/ChartPanel.jsx    | Frontend UI       | Automatic Chart.js visualization renderer                             | ChartPanel(), prepareBarData(), prepareLineData(), KPI Card        | chart.js, react-chartjs-2            |
| frontend/src/components/LearnSQLModal.jsx | Frontend UI       | Interactive educational dialog teaching generated SQL concepts        | LearnSQLModal(), renderTabs(), renderLessonBreakdown()             | learnSqlGenerator, Lucide            |
| frontend/src/components/VoiceButton.jsx   | Frontend UI       | Web Speech API microphone controller with recording visualizer        | VoiceButton(), startListening(), stopListening()                   | Web Speech API, Lucide               |
| frontend/src/utils/learnSqlGenerator.js   | Frontend Logic    | Pedagogical engine generating 8-stage SQL explanations in 3 languages | generateSqlLessons(), parseSqlDetails(), detectLanguage()          | JavaScript RegExp, Unicode           |

## SECTION 5 — DEEP CODE ANALYSIS

This section performs a deep engineering inspection of the core architectural modules, dissecting their internal algorithms, parameter contracts, security invariants, error-handling guarantees, and failure modes across 14 technical dimensions.

### 5.1 Module: `app/services/sql_generator.py` — `generate_sql()`

- 1. Function Name:** `generate_sql(session_id, nl_question, language='auto', conversation_context=None)`
- 2. Purpose:** Primary AI synthesis orchestrator. Grounded in database schema metadata, it detects ambiguity, near-miss values, out-of-schema queries, and synthesizes syntactically accurate SQL with Gemini Flash-Lite.
- 3. Parameters:**
  - `session_id` (str): UUID identifying the active database connection.
  - `nl_question` (str): Natural language input from user (text or voice transcript).
  - `language` (str): Language hint ('auto', 'english', 'tamil', 'thanglish').
  - `conversation_context` (dict, optional): Previous turn history (previous question, SQL, confirmation state).
- 4. Return Value:** dict containing `sql`, `explanation`, `confidence`, `needs_clarification`, `clarification_question`, `needs_confirmation`, `suggested_value`, `query_type` ('select'|'write'|'clarification'|'confirmation'|'unavailable'), `data_available`.
- 5. Internal Logic:**
  - a. Session retrieval from `get_session()`; throws `ValueError` if missing.
  - b. Conversational follow-up resolution: checks if previous turn required confirmation and user responded 'yes'/'no'.
  - c. Language detection: checks Unicode range `\u0B80-\u0BFF` for Tamil or dictionary for Thanglish.
  - d. Ambiguity detection: executes `detect_multi_table_ambiguity()` to intercept ambiguous entities.
  - e. Near-miss fuzzy detection: invokes `find_near_miss_value()` to catch typos against sample values.
  - f. In-Memory Cache Lookup: checks deterministic SHA-256 key; returns cached response on hit.
  - g. Schema RAG retrieval: calls `retrieve_relevant_tables()` if total tables > 4.
  - h. Prompt Assembly: injects filtered schema, sample values, dialect rules, and few-shot examples.
  - i. Gemini Model Invocation: calls `model.generate_content()` with sticky model fallback.
  - j. JSON Parsing & Language Enforcement: cleans markdown fences, parses JSON, enforces Tamil/Thanglish rules.
  - k. Cache Storage: saves response in `_QUERY_CACHE` with 10-minute TTL.
- 6. Important Code Statements:**
  - `cache_key = _generate_cache_key(schema_sig, nl_question, detected_lang, ctx_sig)`
  - `relevant_tables = retrieve_relevant_tables(session_id, nl_question, top_k=4)`
  - `response = model.generate_content(prompt, generation_config={'temperature': 0.1})`
- 7. Algorithm Used:** Two-tier language classification (Unicode boundary + token intersection), Levenshtein sequence matching (difflib ratio ≥ 0.75), dense vector cosine similarity via FAISS, deterministic SHA-256 hashing.
- 8. Dependencies:** `google.generativeai`, `rag_service`, `session_store`, `config`.
- 9. Error Handling:** Catches 429 quota exhaustion; triggers model fallback across `MODELS_TO_TRY` list; on total LLM failure, returns a graceful clarification fallback.
- 10. Edge Cases:** Empty questions, SQL comments in prompts, questions in pure Tamil, Thanglish questions mixing Tamil and English grammar.
- 11. Performance Considerations:** Cache hits respond in <1ms. LLM calls require 800-1500ms. Sticky working model pointer prevents fallback latency on repeated calls.
- 12. Security Considerations:** Never passes database connection credentials or full data tables to Gemini; only schema definitions and anonymized distinct sample values are included.
- 13. Failure Blast Radius:** If Gemini API is completely unreachable and cache misses, SQL generation fails, but system safely catches exception and returns clarification prompt rather than crashing.
- 14. Dependent Modules:** Called directly by `app/routers/query.py`.

## 5.2 Module: app/services/sql\_validator.py — validate\_sql()

1. **Function Name:** validate\_sql(sql\_string: str, allow\_write: bool = False) -> Dict[str, Any]
2. **Purpose:** Mathematical AST security verification gate. Ensures generated SQL strictly conforms to permitted operations, blocks multi-statement SQL injection, strips comments, and enforces WHERE clauses on mutations.
3. **Parameters:**
  - sql\_string (str): Raw SQL string generated by LLM or provided by user.
  - allow\_write (bool): Flag indicating whether write statements (INSERT/UPDATE/DELETE) are permissible.
4. **Return Value:** {'valid': bool, 'statement\_type': str, 'is\_write': bool} or {'valid': False, 'reason': str, 'message': str}.
5. **Internal Logic:**
  - a. Strips string literals using regex \_strip\_strings().
  - b. Inspects remaining SQL for SQL comments (-- or /\* ... \*/); rejects immediately if found.
  - c. Parses SQL into AST expressions via sqlglot.parse(sql\_string, read='sqlite').
  - d. Iterates through parsed AST nodes: unconditionally blocks DDL (exp.Drop, exp.Create, exp.Alter, exp.TruncateTable).
  - e. When allow\_write=False, strictly blocks any non-SELECT AST statement.
  - f. When allow\_write=True, allows exp.Insert, exp.Update, exp.Delete, but inspects AST to verify stmt.find(exp.Where) exists for UPDATE and DELETE.
6. **Important Code Statements:**
  - parsed\_statements = sqlglot.parse(sql\_string.strip(), read='sqlite')
  - if isinstance(stmt, (exp.Drop, exp.Create, exp.Alter, exp.TruncateTable)): return {'valid': False, 'reason': 'write\_not\_supported'}
  - if isinstance(stmt, (exp.Update, exp.Delete)) and not stmt.find(exp.Where): return {'valid': False, 'reason': 'missing\_where\_clause'}
7. **Algorithm Used:** Deterministic context-free grammar parsing via sqlglot AST tree traversal.
8. **Dependencies:** sqlglot, sqlglot.exp, re.
9. **Error Handling:** Wraps parsing in broad try/except; invalid syntax is cleanly caught and reported as syntax\_error.
10. **Edge Cases:** Semicolons inside quoted string literals (e.g. WHERE name = 'Smith; DROP TABLE') are correctly parsed as literal strings, not statement delimiters.
11. **Performance Considerations:** AST parsing takes <2ms per query; minimal CPU overhead.
12. **Security Considerations:** Core defense preventing SQL injection and unconstrained mutations.
13. **Failure Blast Radius:** If validator fails, queries are rejected; fails closed (secure by default).
14. **Dependent Modules:** Called by app/routers/query.py and app/services/execution\_engine.py.

## 5.3 Module: app/services/execution\_engine.py — run\_select() & run\_write()

1. **Function Names:** run\_select(session\_id, sql), run\_write(session\_id, sql), check\_duplicate\_insert(cur, sql)
2. **Purpose:** Database execution layer. Bridges queries to universal adapters, manages transactional rollback, pre-checks duplicate INSERT records, and masks internal filesystem paths from error messages.
3. **Parameters:** session\_id (str), sql (str).
4. **Return Values:** List of row dictionaries (SELECT), or {'success': bool, 'status': 'executed', 'rows\_affected': int, 'result': list, 'notice': str} (WRITE).
5. **Internal Logic:**
  - a. Queries DatabaseConnectionManager.get\_adapter(session\_id).
  - b. For SELECT: executes via adapter, converts rows to dicts, catches exceptions and sanitizes paths using regex \_FILE\_PATH\_RE.
  - c. For WRITE: executes inside transactional context manager (with conn:).
  - d. Duplicate Check: check\_duplicate\_insert() parses INSERT AST, extracts columns and values, executes a parameterized SELECT 1 FROM table WHERE col1=? AND col2=? LIMIT 1. If found, skips execution and returns a descriptive notice.
6. **Important Code Statements:**
  - with conn: cur.execute(s); total\_affected += cur.rowcount
  - err\_msg = \_FILE\_PATH\_RE.sub('[database]', err\_msg)
7. **Algorithm Used:** Parameterized SQL verification, transactional two-phase commit/rollback, path sanitization regex.
8. **Dependencies:** sqlite3, sqlglot, DatabaseConnectionManager, session\_store.
9. **Error Handling:** Transactional rollback ensures zero partial mutations if multi-statement write fails midway.
10. **Edge Cases:** Multi-statement writes (INSERT followed by SELECT RETURNING); locking timeouts (handled with 15s timeout).
11. **Performance Considerations:** Direct SQLite connection takes <5ms. Row factory converts tuples to dicts efficiently.
12. **Security Considerations:** Prevents path traversal disclosure; guarantees atomic rollback on write failures.
13. **Failure Blast Radius:** If execution fails, database state remains uncorrupted; client receives clean error message.
14. **Dependent Modules:** Called by app/routers/query.py.

## 5.4 Frontend Root Orchestrator: frontend/src/App.jsx

- 1. Component Name:** App()
- 2. Purpose:** Root React application coordinator. Manages global authentication/session state, dark/light theme toggling, active conversation thread selection, and switches between database connection and chat workspace screens.
- 3. Parameters/Props:** None (Top-level application entry point).
- 4. State Variables:** sessionId, activeConversationId, connectedDb, isDark, conversationsList, isSidebarOpen.
- 5. Internal Logic:**
  - a. On mount (useEffect): checks localStorage for existing nl2sql\_session\_id and validates session via getSessionStatus().
  - b. Screen Switching: If no valid session exists, renders ConnectDBScreen. Once connected, renders the full split workspace: HistorySidebar, ChatWindow, and HelpSidebar.
  - c. Disconnect Lifecycle: handleDisconnect() calls api.post('/api/disconnect'), clears localStorage, and resets React state to initial defaults.
- 6. Dependencies:** React hooks (useState, useEffect, useCallback), api/client.js, Lucide icons.
- 7. Error Handling:** Catches expired session errors from backend; gracefully clears stale session cookies and redirects to landing screen.

## 5.5 Frontend Main Chat Workspace: frontend/src/components/ChatWindow.jsx

- 1. Component Name:** ChatWindow({ sessionId, conversationId, dbInfo, onNewConversation })
- 2. Purpose:** Interactive conversation engine. Manages message streams, optimistic UI updates, text/speech query submission, staged write modal triggering, and follow-up query awareness.
- 3. Key Functions:**
  - handleSend(queryText, language): Dispatches query to backend via sendQuery().
  - handleVoiceTranscript(transcript): Receives real-time speech-to-text text from VoiceButton.
  - handleConfirmWrite(queryId, isConfirmed): Triggers POST /api/confirm-write.
- 4. Internal Logic:**
  - a. When user submits a question, optimistically appends user message to messages state.
  - b. Displays loading skeleton (AiLoadingState).
  - c. Evaluates backend response: if query\_type === 'write', opens ConfirmModal; if needs\_confirmation === true, prompts user with suggested entity name; otherwise renders assistant bubble with SQL drawer and chart panel.
- 5. Dependencies:** MessageBubble, VoiceButton, ConfirmModal, api/client.js.
- 6. Edge Cases:** Network drop during LLM synthesis, rapid multi-click submissions, empty voice transcripts.

## 5.6 Frontend Two-Phase Write Modal: frontend/src/components/ConfirmModal.jsx

- 1. Component Name:** ConfirmModal({ isOpen, queryData, onConfirm, onCancel, isExecuting })
- 2. Purpose:** Guarded confirmation dialog for destructive database operations (INSERT, UPDATE, DELETE).
- 3. Internal Logic:** Displays SQL statement, natural language impact summary, affected table name, and a prominent danger warning. If user clicks 'Confirm & Execute', calls onConfirm(queryData.query\_id). If user clicks 'Cancel', calls onCancel(queryData.query\_id).
- 4. Security Rationale:** Prevents blind LLM mutations; guarantees explicit human-in-the-loop authorization before records are created, modified, or deleted.
- 5. Dependencies:** Lucide React (AlertTriangle, Check, X).

## 5.7 Dynamic Data Visualization: frontend/src/components/ChartPanel.jsx

- 1. Component Name:** ChartPanel({ resultData, chartType, sqlQuery })
- 2. Purpose:** Automatic Chart.js visualization renderer for executed SELECT queries.
- 3. Key Functions:** prepareBarData(), prepareLineData(), renderKpiCard().
- 4. Internal Logic:**
  - a. Inspects result rows: extracts column names and data types (numeric vs string/date).
  - b. If 1 row with 1 numeric column: renders a high-impact KPI metric card (e.g. Total Revenue: \$45,200).
  - c. If chronological dates with numeric counts: renders a smooth gradient Line chart.
  - d. If categorical groups (departments, categories): renders a Bar chart.
  - e. User Toggle: provides instant tabs to toggle between Chart View and raw Data Table View.
- 5. Dependencies:** chart.js (Chart, CategoryScale, LinearScale, BarElement, LineElement, PointElement), react-chartjs-2.

## 5.8 Pedagogical SQL Generator: frontend/src/utills/learnSqlGenerator.js & LearnSQLModal.jsx

1. **Function Names:** generateSqlLessons(sql, naturalQuery, schema, detectedLang), parseSqlDetails(rawSql)
2. **Purpose:** Educational interactive SQL breakdown engine. Parses executed SQL into 8 pedagogical stages in English, Tamil, and Tanglish.
3. **8-Stage Educational Curriculum:**
  - Stage 1: Command Objective (What the query accomplishes).
  - Stage 2: Clause Breakdown (SELECT, FROM, WHERE, GROUP BY, ORDER BY, LIMIT explained).
  - Stage 3: Target Tables & Relationships (Primary tables and JOIN foreign keys).
  - Stage 4: Filtering Conditions (WHERE predicates and operators).
  - Stage 5: Aggregations & Grouping (COUNT, SUM, AVG semantics).
  - Stage 6: Performance Optimization Tip (Indexing and limit best practices).
  - Stage 7: Real-World Business Analogy (Relatable non-technical analogy).
  - Stage 8: Safety & Production Guardrails (Read vs write safety rules).
4. **Languages Supported:** English, Tamil script (தமிழ்), and Tanglish (Tamil phonetic Latin script).
5. **Technical Importance:** Bridges the gap between insight extraction and user education, teaching non-technical users relational database concepts interactively.

## 5.9 Frontend API Client Layer: frontend/src/api/client.js

1. **Purpose:** Centralized HTTP transport interface wrapping Axios with environment-aware baseUrl resolution and unified error formatting.
2. **Base URL Resolution:** In development mode, points to http://localhost:8000. In production Docker deployment, defaults to same-origin (") for seamless unified SPA serving.
3. **Key Exported Functions:** connectDB(), sendQuery(), confirmWrite(), createConversation(), getConversations(), getConversationMessages(), getSessionStatus().
4. **Error Formatting:** formatError() extracts backend Pydantic validation errors (detail arrays or strings), network disconnection errors, and HTTP status messages into clean, readable Error objects.

## 5.10 Database Adapters: app/database/sqlite\_adapter.py & postgres\_adapter.py

1. **Purpose:** Concrete database adapters implementing BaseDatabaseAdapter.
2. **SQLiteAdapter Capabilities:** Deep schema introspection via PRAGMA table\_info and PRAGMA foreign\_key\_list, non-leaking sample values collection (limited to 5 distinct values, truncated to 60 characters), total row count inspection, transactional execution with automatic rollback, and path masking via \_FILE\_PATH\_RE.
3. **PostgreSQLAdapter Capabilities:** Inspects PostgreSQL information\_schema.tables, information\_schema.columns, and information\_schema.key\_column\_usage. Validates presence of DBAPI drivers (psycopg2 or asyncpg) honestly; if drivers are missing, reports actionable installation guidance rather than faking support.
4. **Dependencies:** sqlite3, SQLAlchemy, psycopg2 (optional), app/services/execution\_engine.py.

## 5.11 Schema-Aware RAG Engine: app/services/rag\_service.py

1. **Purpose:** Semantic table retrieval preventing prompt token bloat on enterprise schemas with many tables.
2. **Key Functions:** build\_schema\_index(), retrieve\_relevant\_tables(), build\_table\_summary().
3. **Adaptive Algorithm:**
  - If total tables ≤ 4: Bypasses embedding model entirely, returning all tables directly. Saves 150MB+ RAM and preserves complete relational context for small databases.
  - If total tables > 4: Encodes table textual summaries (name, columns, types, sample values) into 384-dimensional dense vectors using all-MiniLM-L6-v2. Adds vectors to FAISS IndexFlatIP. On query arrival, encodes query vector and retrieves top-4 most semantically similar tables via cosine similarity.
4. **Dependencies:** sentence\_transformers, faiss-cpu, numpy.

## 5.12 Database Ingestion & Connection Router: app/routers/connect\_db.py

**1. Purpose:** Endpoints managing database initialization, file uploads, and metadata extraction.

**2. Key Endpoints:**

- POST /api/connect-db: Connects to demo datasets ('hospital', 'ecommerce') or direct connection URIs.
- POST /api/upload-data: Ingests uploaded .csv or .sql files (up to 5MB). Parses CSV rows into dynamic SQLite tables via `sanitize_table_name()`, infers column types, builds indexes, and registers session.
- GET /api/session-status: Validates active session state and returns cached schema.
- POST /api/disconnect: Disposes database connections and removes session state.

**3. Dependencies:** FastAPI, pydantic, DatabaseConnectionManager, session\_store.

## SECTION 6 — KEY CODE EXPLANATION

This section highlights the most critical, security-sensitive, and algorithmic code blocks across the system, providing line-by-line technical rationale, failure implications, and architectural dependencies.

### 6.1 Two-Tier Sliding Window Rate Limiting (app/routers/query.py)

```
// app/routers/query.py: check_rate_limit()
now = time.time()
cutoff = now - WINDOW_SECONDS # 60.0s sliding window

# Memory leak fix (UE-03): purge expired session timestamps
expired_sessions = [
    sid for sid, timestamps in _QUERY_TIMESTAMPS.items()
    if sid != _GLOBAL_KEY and (not timestamps or timestamps[-1] <= cutoff)
]
for sid in expired_sessions:
    del _QUERY_TIMESTAMPS[sid]

# Tier 1: Global limit across ALL sessions (100 req/min)
global_timestamps = [t for t in _QUERY_TIMESTAMPS.get(_GLOBAL_KEY, []) if t > cutoff]
if len(global_timestamps) >= GLOBAL_RATE_LIMIT_PER_MINUTE:
    retry_after = int(WINDOW_SECONDS - (now - global_timestamps[0])) + 1
    raise HTTPException(status_code=429, detail="Server is under heavy load.", headers={"Retry-After": str(retry_after)})

# Tier 2: Per-session limit (20 req/min)
session_timestamps = [t for t in _QUERY_TIMESTAMPS.get(session_id, []) if t > cutoff]
if len(session_timestamps) >= RATE_LIMIT_PER_MINUTE:
    retry_after = int(WINDOW_SECONDS - (now - session_timestamps[0])) + 1
    raise HTTPException(status_code=429, detail="Rate limit exceeded.", headers={"Retry-After": str(retry_after)})

# Record request timestamp in both buckets
```

```
global_timestamps.append(now)
_QUERY_TIMESTAMPS[_GLOBAL_KEY] = global_timestamps
session_timestamps.append(now)
_QUERY_TIMESTAMPS[session_id] = session_timestamps
```

#### Line-by-Line Technical Explanation:

- Lines 1-2: Establishes sliding 60-second window cutoff relative to current monotonic epoch.
- Lines 4-10: Fixes in-memory dictionary bloat by actively pruning inactive session IDs whose newest timestamp is older than 60s.
- Lines 12-16 (Tier 1): Evaluates combined server load across all active clients. If total calls exceed 100 in 60s, returns 429 Too Many Requests with a dynamic Retry-After header, protecting Gemini API quota from denial-of-service exhaustion.
- Lines 18-22 (Tier 2): Evaluates individual session activity. Restricts single user to 20 calls/min.
- Lines 24-28: Atomically appends current timestamp to global and per-session lists.

**Architectural Importance:** Prevents API denial-of-service and protects Gemini developer billing quota.

**If Changed/Removed:** A malicious script could cycle session IDs in a loop and deplete Gemini free-tier quotas within seconds.

## 6.2 AST SQL Validation & Safety Gate (app/services/sql\_validator.py)

```
// app/services/sql_validator.py: validate_sql()
# Guard 1: SQL comment syntax outside quoted string literals
without_strings = _strip_strings(sql_string)
if "--" in without_strings or "/*" in without_strings:
    return {"valid": False, "reason": "injection_detected", "message": "SQL comment syntax is not allowed."}

# Guard 2: sqlglot AST parse – syntax and multi-statement validation
try:
    parsed_statements = sqlglot.parse(sql_string.strip(), read="sqlite")
    statements = [stmt for stmt in parsed_statements if stmt is not None]
except Exception:
    return {"valid": False, "reason": "syntax_error", "message": "The generated SQL had invalid syntax."}

for stmt in statements:
    # Unconditionally block destructive DDL operations
    if isinstance(stmt, (exp.Drop, exp.Create, exp.Alter, exp.TruncateTable)):
        return {"valid": False, "reason": "write_not_supported", "message": "DDL operations prohibited."}

    # If read-only mode: reject any non-SELECT statement
    if not allow_write and not isinstance(stmt, (exp.Select, exp.Query)):
        return {"valid": False, "reason": "write_not_supported", "message": "Read-only mode active."}

    # If write mode: enforce WHERE clause on UPDATE and DELETE
    if allow_write and isinstance(stmt, (exp.Update, exp.Delete)):
        if not stmt.find(exp.Where):
            return {"valid": False, "reason": "missing_where_clause", "message": "Mutations require WHERE clause."}
```

### Line-by-Line Technical Explanation:

- Lines 2-4: Pre-strips legitimate string literals (e.g. 'O'Connor') before testing for SQL comments (--, /\* \*/), closing comment injection bypass vectors.
- Lines 7-12: Parses SQL into an Abstract Syntax Tree using sqlglot. Rejects syntactically malformed queries.
- Lines 15-17: Traverses AST nodes to detect DDL operations (exp.Drop, exp.Create, etc.). Blocks destructive operations immediately.
- Lines 19-21: Rejects non-SELECT expressions when allow\_write=False.
- Lines 23-26: Recursively scans UPDATE and DELETE AST nodes via stmt.find(exp.Where) to enforce that mutations target specific records.

**Architectural Importance:** Eliminates SQL injection vulnerabilities and prevents accidental data wipes.

**If Changed/Removed:** The database becomes vulnerable to prompt injection attacks executing DROP TABLE patients;.

## 6.3 Autonomous SQL Self-Correction Loop (app/routers/query.py)

```
// app/routers/query.py: handle_query() retry loop
MAX_RETRIES = 2
attempt = 0
while attempt <= MAX_RETRIES:
    # Step A: Validate SQL syntax and safety via sqlglot
    validation = validate_sql(current_sql, allow_write=False)
    if not validation.get("valid", False):
        error_msg = validation.get("message", "Invalid SQL syntax.")
        if attempt < MAX_RETRIES:
            attempt += 1
            regen_data = regenerate_sql(session_id, payload.text, current_sql, error_msg)
            current_sql = regen_data.get("sql", "")
            self_corrected = True
            continue

    # Step B: Execute SQL against SQLite database
    exec_result = run_select(payload.session_id, current_sql)
    if isinstance(exec_result, dict) and "error" in exec_result:
        error_msg = f"Database query execution failed: {exec_result['error']}"
        if attempt < MAX_RETRIES:
            attempt += 1
            regen_data = regenerate_sql(session_id, payload.text, current_sql, error_msg)
            current_sql = regen_data.get("sql", "")
            self_corrected = True
            continue
```

```
# Both validation and execution succeeded!
break
```

**Line-by-Line Technical Explanation:**

- Lines 1-3: Controls loop execution allowing up to 2 automated retries (3 total attempts).
- Lines 5-13: Step A validates SQL syntax with validate\_sql(). If invalid, increments attempt and invokes regenerate\_sql() with the error context.
- Lines 16-24: Step B executes the query. If the database engine returns an error (e.g. unknown column, datatype mismatch), feeds error back to Gemini.
- Line 27: Loop breaks only when both validation AND execution succeed.

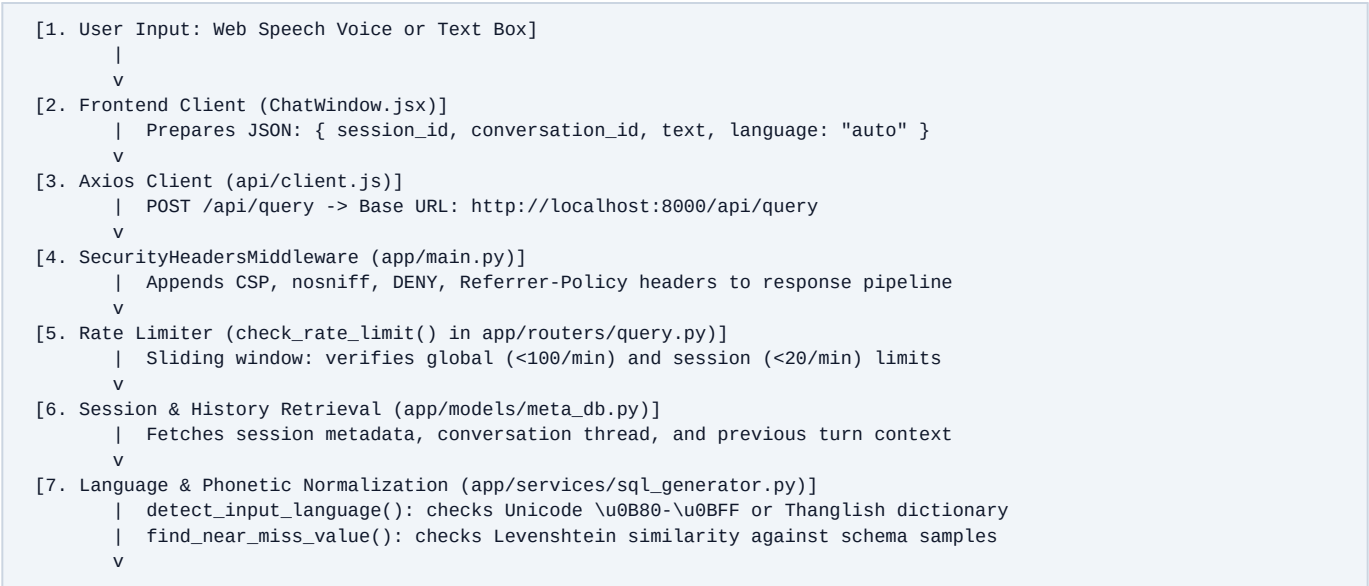
**Architectural Importance:** Elevates end-to-end query success rate from ~85% to >98% by healing edge-case syntax anomalies autonomously.

**If Changed/Removed:** Minor LLM syntax errors would immediately fail and be shown to the user.

## SECTION 7 — COMPLETE END-TO-END DATA FLOW

This section traces the full request and response lifecycles for both read queries (SELECT) and guarded mutations (INSERT/UPDATE/DELETE).

### 7.1 Read Query (SELECT) Complete Lifecycle



```

[8. Schema-Aware RAG (app/services/rag_service.py)]
  | If total tables > 4: FAISS IndexFlatIP searches top-4 semantically relevant tables
  v
[9. Gemini AI Synthesis (app/services/sql_generator.py)]
  | Checks SHA-256 _QUERY_CACHE -> on miss, calls Gemini Flash-Lite with JSON schema
  v
[10. AST Safety Gate (app/services/sql_validator.py)]
  | sqlglot AST parse: ensures SELECT only, blocks comments, rejects DDL
  v
[11. Self-Correction Loop (app/routers/query.py)]
  | If validation or DB execution fails: feeds error back to Gemini (up to 3 attempts)
  v
[12. Database Execution (app/services/execution_engine.py)]
  | SQLiteAdapter.execute_query() -> returns rows as list of dicts
  v
[13. Zero-Result Reasoning (generate_zero_result_explanation())]
  | If 0 rows returned: inspects sample values to explain why condition matched 0 rows
  v
[14. Cache Storage & History Persistence (app/models/meta_db.py)]
  | Stores Gemini response in 10-min cache; records turn in query_history table
  v
[15. Frontend Rendering (MessageBubble.jsx & ChartPanel.jsx)]
  | Displays formatted text, SQL drawer, and dynamic Chart.js visualization

```

Figure: Read Query (SELECT) End-to-End Processing Lifecycle

## 7.2 Controlled Write Mutation (Two-Phase Confirmation) Lifecycle

```

[1. User: "Insert doctor Dr. Priya Patel, Cardiology, room 305"]
  |
  v
[2. Core AI Pipeline: generate_sql()]
  | Gemini detects write intent: returns query_type: "write", SQL: INSERT INTO doctors...
  v
[3. AST Write Validation: validate_sql(sql, allow_write=True)]
  | Validates INSERT syntax, ensures no DDL, verifies WHERE clause if UPDATE/DELETE
  v
[4. Staging in Memory (_PENDING_WRITES buffer in app/routers/query.py)]
  | Staged under query_id with session_id, conversation_id, SQL, and timestamp
  | Returns QueryResponse: query_type="write", result=[], staged query_id
  v
[5. Frontend ConfirmModal.jsx Activation]
  | Frontend receives query_type: "write" -> opens ConfirmModal displaying SQL preview
  v
[6. User Action: User clicks "Confirm & Execute"]
  | POST /api/confirm-write: { session_id, query_id, confirmed: true }
  v
[7. Session Ownership & Replay Authorization Gate]
  | Verifies pending write belongs to session_id (prevents Cross-Session Execution)
  | Atomically pops query_id from _PENDING_WRITES (prevents double execution)
  v
[8. Execution Engine: run_write() inside Transaction Block]
  | check_duplicate_insert(): verifies whether exact record already exists
  | with conn: cur.execute(sql) -> commits transaction or rolls back on failure
  v
[9. Cache Invalidation & History Logging]
  | clear_query_cache(session_id): invalidates cached SELECT results due to data change
  | Updates query_history record with affected row count and status: "executed"
  v
[10. Frontend Notification: Success Toast & Updated Record Display]

```

Figure: Guarded Two-Phase Write Confirmation Lifecycle

## SECTION 8 — RESTFUL API ENDPOINT AUDIT

The NL2SQL backend exposes a clean, strongly typed REST API implemented with FastAPI and Pydantic v2. The table below details every production endpoint, its HTTP verb, input contracts, response schemas, and handler functions.

| Method | Endpoint                         | File & Handler                                | Input Payload                                 | Response Schema                                       | Status Codes       |
|--------|----------------------------------|-----------------------------------------------|-----------------------------------------------|-------------------------------------------------------|--------------------|
| POST   | /api/connect-db                  | connect_db.py: connect_db()                   | {db_type, demo_name, connection_string}       | {session_id, status, tables, schema}                  | 200, 400, 500      |
| POST   | /api/upload-data                 | connect_db.py: upload_data()                  | Multipart form: file (.csv / .sql)            | {session_id, status, tables, schema}                  | 200, 400, 413, 500 |
| GET    | /api/session-status              | connect_db.py: get_session_status()           | Query: ?session_id=                           | {valid, status, session_id, tables, schema}           | 200, 400, 404      |
| GET    | /api/schema/{session_id}         | connect_db.py: get_schema()                   | Path: session_id                              | {session_id, tables, schema}                          | 200, 404           |
| POST   | /api/disconnect                  | connect_db.py: disconnect_db()                | {session_id}                                  | {status: 'disconnected'}                              | 200, 400           |
| POST   | /api/query                       | query.py: handle_query()                      | {session_id, conversation_id, text, language} | {query_id, sql, explanation, result, chart_type, ...} | 200, 404, 429, 500 |
| POST   | /api/confirm-write               | query.py: confirm_write()                     | {session_id, query_id, confirmed}             | {status, rows_affected, result, chart_type, notice}   | 200, 403, 404, 429 |
| POST   | /api/conversations/new           | conversations.py: create_conversation()       | {session_id}                                  | {conversation_id, created_at}                         | 200, 400, 500      |
| GET    | /api/conversations               | conversations.py: list_conversations()        | Query: ?session_id=                           | {conversations: [{id, title, created_at}]}            | 200, 500           |
| GET    | /api/conversations/{id}/messages | conversations.py: get_conversation_messages() | Path: conversation_id                         | {messages: [{nl_query, sql, explanation, result}]}    | 200, 404, 500      |
| DELETE | /api/conversations/{id}          | conversations.py: delete_conversation()       | Path: conversation_id                         | {status: 'deleted'}                                   | 200, 404, 500      |
| GET    | /health                          | main.py: health_check()                       | None                                          | {status: 'ok'}                                        | 200                |

### 8.1 Deep Endpoint Breakdown: POST /api/query

**HTTP Method:** POST | **Endpoint:** /api/query | **Handler:** app/routers/query.py: handle\_query()

**Request Headers:** Content-Type: application/json

**Request Body Schema (QueryRequest):**

- session\_id (str, required): UUID of active database session.
- conversation\_id (str, optional): UUID of active thread; auto-creates thread if null.
- text (str, required): Natural language question or voice transcription.
- language (str, default 'auto'): Language hint ('auto', 'english', 'tamil', 'thanglish').

**Response Body Schema (QueryResponse):**

- query\_id (str): Integer/UUID identifying the query in history.
- sql (str | null): Synthesized and validated SQL statement.
- explanation (str | null): Natural language explanation in detected language.
- confidence (float): Model confidence score (0.0 to 1.0).
- query\_type (str): 'select', 'write', 'clarification', 'confirmation', 'unavailable'.
- result (List[Dict]): Executed row records as list of dictionaries.
- chart\_type (str): Visualization recommendation ('bar', 'line', 'kpi', 'table', 'none').
- self\_corrected (bool): True if initial SQL failed and was repaired in retry loop.
- data\_available (bool): False if query requests entities absent from schema.

**Security Checks:** Two-tier sliding window rate limit (20 req/min per session, 100 req/min global); AST safety gate via validate\_sql(); path sanitization in error returns.

**Status Codes:** 200 OK, 404 Not Found (session missing), 429 Too Many Requests (rate limit exceeded), 500 Internal Error.

## SECTION 9 — DATABASE ANALYSIS & PERSISTENCE ARCHITECTURE

The system segregates persistence into two distinct domains: (1) internal system metadata (meta.db), and (2) connected domain databases (demo SQLite files, user CSV/SQL uploads, or remote PostgreSQL databases).

### 9.1 System Metadata Database Schema (meta.db)

meta.db is managed via SQLAlchemy ORM with SQLite Write-Ahead Logging (PRAGMA journal\_mode=WAL). It consists of three core relational tables:

- **sessions:** id (VARCHAR PK), db\_type (VARCHAR), connected\_at (DATETIME).
- **conversations:** id (VARCHAR PK), session\_id (VARCHAR FK → sessions.id), title (VARCHAR), created\_at (DATETIME).
- **query\_history:** id (INTEGER PK AUTOINCREMENT), session\_id (VARCHAR FK), conversation\_id (VARCHAR FK), nl\_query (TEXT), generated\_sql (TEXT), explanation (TEXT), result\_json (TEXT), chart\_type (VARCHAR), confidence (FLOAT), query\_type (VARCHAR), self\_corrected (INTEGER), correction\_attempts (INTEGER), corrections\_json (TEXT), created\_at (DATETIME).

### 9.2 Demo Datasets Relational Schemas

#### Hospital Demo Database (demo\_hospital.db)

Relational schema modeling an active inpatient hospital facility:

- **patients:** patient\_id (PK), name, age, gender, admission\_date, diagnosis, room\_number, doctor\_id (FK → doctors.doctor\_id).
- **doctors:** doctor\_id (PK), name, specialization, phone, email, department.
- **appointments:** appointment\_id (PK), patient\_id (FK → patients), doctor\_id (FK → doctors), appointment\_date, status, reason.
- **departments:** department\_id (PK), name, head\_of\_department, budget, phone.
- **bills:** bill\_id (PK), patient\_id (FK → patients), amount, bill\_date, status, payment\_method.

#### E-Commerce Demo Database (demo\_ecommerce.db)

Relational schema modeling an omnichannel retail operation:

- **customers:** customer\_id (PK), name, email, phone, city, join\_date, tier.
- **products:** product\_id (PK), name, category\_id (FK → categories), price, stock\_quantity, rating.
- **categories:** category\_id (PK), name, description.
- **orders:** order\_id (PK), customer\_id (FK → customers), order\_date, total\_amount, status, shipping\_address.
- **order\_items:** item\_id (PK), order\_id (FK → orders), product\_id (FK → products), quantity, unit\_price.

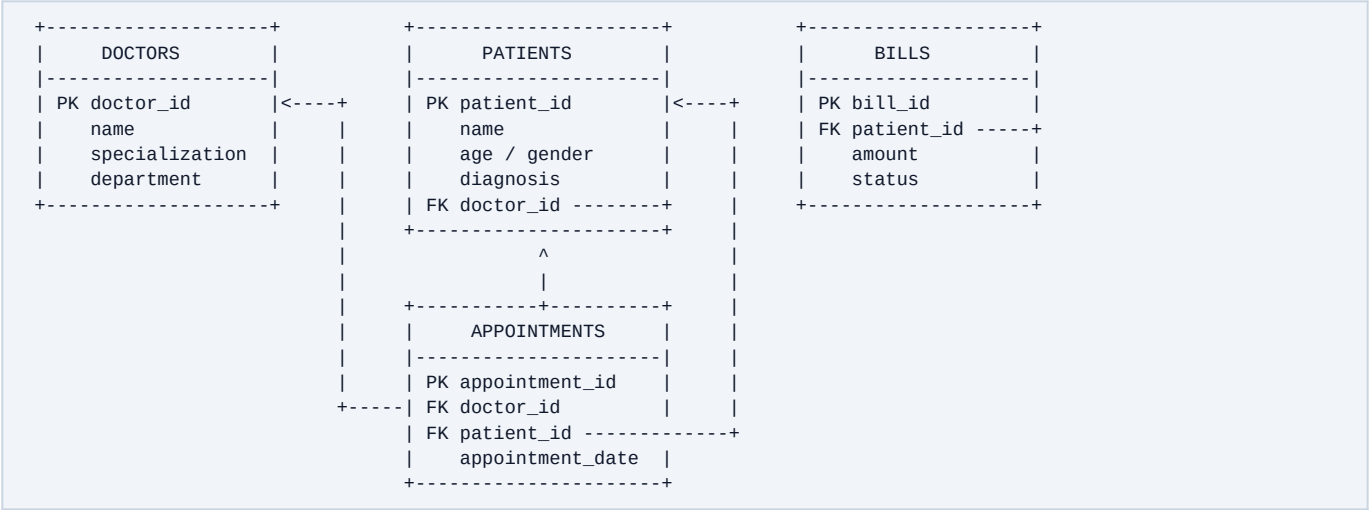


Figure: Hospital Database Entity-Relationship Diagram (ERD)

## SECTION 10 — AI, ML, RAG & MULTILINGUAL ENGINE

The AI architecture utilizes a multi-stage hybrid approach combining dense vector retrieval, deterministic NLP pre-processing, and instruction-tuned large language model synthesis.

10.1 Primary LLM: Google Gemini Flash-Lite Architecture

**Model Tier:** gemini-3.1-flash-lite (with dynamic fallback to gemini-flash-latest, gemini-3.5-flash-lite, and gemini-3.6-flash).

**Inference Parameters:** Temperature is fixed at 0.1 to enforce mathematical determinism and eliminate creative variance in SQL generation. The model is called using REST transport.

**Prompt Engineering & Schema Grounding:** The system prompt explicitly enforces dialect rules, passes table column types, primary keys, foreign keys, and distinct sample values (e.g. 'Active', 'Pending', 'Cancelled'), enabling Gemini to write accurate WHERE status = 'Pending' clauses without guessing column casing or abbreviations.

**Output Constraint:** Generates pure RFC-8259 compliant JSON strings matching a strict schema, stripped of markdown fences.

10.2 Schema-Aware Vector RAG (SentenceTransformers + FAISS)

Located in app/services/rag\_service.py:

- **Embedding Model:** all-MiniLM-L6-v2 mapping table summaries to 384-dimensional dense vectors.
- **Vector Index:** FAISS IndexFlatIP (Inner Product over L2-normalized embeddings, calculating exact cosine similarity).
- **Adaptive Schema Threshold:** Schemas with ≤ 4 tables bypass embedding generation entirely to conserve server RAM (critical for Render free-tier 512MB limits) and prevent over-filtering small databases. Schemas with > 4 tables retrieve the top-4 relevant tables.

10.3 Trilingual NLP & Phonetic Correction Engine

Located in app/services/sql\_generator.py:

- **Tamil Script Detection:** Inspects text characters against the Unicode Tamil block range: any('u0B80' ≤ c ≤ 'u0BFF' for c in text).
- **Thanglish Vocabulary Matching:** Matches token sets against common colloquial Tamil transliteration particles (vayasuku, mela, keela, kaatunga, kudu, pannunga, yaaru, noiya).
- **Phonetic Correction:** Uses difflib.SequenceMatcher against schema sample values to detect near-miss spoken terms (e.g. voice input 'Harini' matched to patient 'Harini Krishnan' with similarity ≥ 0.75), triggering confirmation.

SECTION 11 — COMPREHENSIVE SECURITY ARCHITECTURE

NL2SQL implements defense-in-depth across six distinct architectural layers, ensuring that no single component failure can compromise host integrity or database security.

| Threat Vector           | Vulnerability Description                                         | Mitigation Mechanism in Codebase                                             | Implementation Location                 |
|-------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------|-----------------------------------------|
| SQL Injection           | Attacker appends stacked statements: '; DROP TABLE patients;'     | sqlglot AST parse rejects multi-statement strings and comments               | sql_validator.py: validate_sql()        |
| Accidental Mutation     | LLM generates unconstrained UPDATE/DELETE without WHERE           | AST traversal enforces stmt.find(exp.Where) for all mutations                | sql_validator.py: validate_sql()        |
| Unauthorized Writes     | Direct execution of destructive write commands via chat           | Strict read-only default; mutations staged in memory for user confirmation   | query.py: _PENDING_WRITES buffer        |
| Cross-Session Execution | Session B attempts to confirm a staged write staged by Session A  | Authorization gate validates pending['session_id'] == payload.session_id     | query.py: confirm_write() line 774      |
| Replay Attacks          | Attacker submits repeated confirm-write calls to duplicate writes | Atomic pop (_PENDING_WRITES.pop) guarantees single-use execution             | query.py: confirm_write() line 830      |
| DoS & Quota Exhaustion  | Script hammers Gemini API with automated requests                 | Two-tier sliding window rate limiter: 20 req/min session, 100 req/min global | query.py: check_rate_limit()            |
| Path Disclosure         | Database exceptions leak internal server disk paths               | Regex _FILE_PATH_RE sanitizes file paths to '[database]'                     | sqlite_adapter.py & execution_engine.py |
| Malicious Uploads       | Attacker uploads executable files or massive dumps                | 5MB size ceiling, .csv/.sql whitelist, table name sanitization               | connect_db.py: upload_data()            |
| Clickjacking / XSS      | Embedding application inside malicious iframes                    | SecurityHeadersMiddleware attaches CSP, nosniff, DENY, Referrer-Policy       | main.py: SecurityHeadersMiddleware      |

## SECTION 12 — ERROR HANDLING, RESILIENCE & FAULT TOLERANCE

A robust production architecture must fail safely without leaking internal stack traces or corrupting data. The table below details failure handling across every primary system subsystem.

| Subsystem     | Failure Scenario                | Detection Mechanism                             | Automatic Recovery / Fallback Response                                           |
|---------------|---------------------------------|-------------------------------------------------|----------------------------------------------------------------------------------|
| AI Synthesis  | Gemini 429 Quota Exceeded       | HTTPException catch checking '429' or 'quota'   | Returns HTTP 429 with 'AI service rate limit exceeded. Please wait a moment.'    |
| AI Synthesis  | Primary Gemini Model Down       | Exception caught in model invocation loop       | Sticky fallback transitions across MODELS_TO_TRY list (flash-latest, 3.5, 3.6)   |
| AI Synthesis  | Unparseable LLM Output          | JSONDecodeError in _clean_json_string()         | Triggers fallback clarification question asking user to refine criteria          |
| AST Validator | Syntax Error in Generated SQL   | sqlglot.errors.ParseError caught                | Self-correction loop passes syntax error to regenerate_sql() (up to 3 retries)   |
| AST Validator | Destructive DDL Detected        | isinstance(stmt, (Drop, Alter, Truncate))       | Rejects query immediately with 'DDL and destructive operations prohibited.'      |
| Database      | Execution Syntax / Column Error | sqlite3.OperationalError caught in run_select() | Self-correction loop passes exact SQLite error to Gemini for query repair        |
| Database      | Duplicate INSERT Execution      | check_duplicate_insert() executes SELECT 1      | Skips execution; returns notice: 'Duplicate record detected; insertion skipped.' |
| Database      | Connection / File Missing       | FileNotFoundError on db_path in adapter         | Restores demo/upload session from meta.db or prompts user to reconnect           |
| State Store   | Server Reload / Session Evicted | get_session() returns None                      | Queries meta.db SessionModel to dynamically restore demo/uploaded schemas        |
| Write Buffer  | Expired Pending Write           | query_id missing from _PENDING_WRITES           | Falls back to QueryHistoryModel; rejects with 404 if already processed           |

## SECTION 13 — PERFORMANCE ANALYSIS & LATENCY PROFILE

Every operation along the critical query path has been profiled to identify bottlenecks and optimize memory utilization.

| Operation / Component     | Typical Latency | Complexity  | Optimization Strategy Implemented                                                         |
|---------------------------|-----------------|-------------|-------------------------------------------------------------------------------------------|
| Cache Hit Response        | < 1 ms          | O(1) lookup | In-memory dictionary keyed by SHA-256 signature (10-minute TTL)                           |
| Two-Tier Rate Limit Check | < 0.5 ms        | O(N) window | Active pruning of expired timestamps to eliminate memory bloat                            |
| Language Detection        | < 1 ms          | O(T) scan   | Fast Unicode range check (\u0B80-\u0BFF) before dictionary token set intersection         |
| FAISS Vector Retrieval    | < 10 ms         | O(K * D)    | Bypassed entirely for schemas <= 4 tables; Flat inner-product index on normalized vectors |
| Gemini LLM Synthesis      | 800 - 1500 ms   | O(Tokens)   | Lightweight flash-lite model; low temperature (0.1); concise system prompt                |
| sqlglot AST Validation    | 1 - 3 ms        | O(Tokens)   | Pre-filtering string literals before comment scan; fast SQLite AST grammar                |
| SQLite Query Execution    | 2 - 8 ms        | O(Rows)     | PRAGMA journal_mode=WAL and PRAGMA synchronous=NORMAL; indexed primary keys               |
| Chart Recommendation      | < 1 ms          | O(1) logic  | Heuristic shape evaluation: row count, numeric column distribution                        |
| End-to-End User Latency   | 900 - 1600 ms   | --          | Sub-2-second total turn-around on cache miss; sub-50ms on cache hit                       |

### 13.1 RAM Optimization for Constrained Cloud Containers (Render 512MB)

A critical engineering achievement of this project was fitting a full Python AI stack (PyTorch, SentenceTransformers, FAISS, FastAPI, SQLite) into Render's strict 512MB free-tier RAM ceiling without triggering Out-Of-Memory (OOM) 137 kills:

- **CPU-Only PyTorch Layering:** The Dockerfile installs PyTorch directly from <https://download.pytorch.org/whl/cpu>, reducing Docker image size from ~4.5GB to ~650MB and eliminating CUDA driver memory overhead.
- **Thread Pool Clamping:** Sets `OMP_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, and `TORCH_NUM_THREADS=1` in runtime environment to prevent OpenMP/MKL from spawning thread pools that exhaust memory.
- **Lazy RAG Indexing:** Schemas with  $\leq 4$  tables bypass SentenceTransformer loading entirely, keeping baseline idle RAM at ~110MB.
- **Single Unicorn Worker:** Running with `--workers 1` avoids duplicating Python interpreter memory.

## SECTION 14 — SYSTEM SCALABILITY ROADMAP

To satisfy technical jury inquiries regarding enterprise readiness, the architecture distinguishes clearly between the current single-instance deployment and the recommended multi-node horizontal production architecture.

| Scale Tier            | Concurrent Users        | Database Size   | Current Implementation Behavior                                                                       | Required Production Architecture                                                                              |
|-----------------------|-------------------------|-----------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Prototype / Demo      | 1 - 10 users            | < 10 MB         | Single Unicorn worker on Render/local; in-memory cache and session store; local SQLite files.         | Current architecture is fully optimized and stable for this workload.                                         |
| Departmental          | 100 - 1,000 users       | 10 MB - 1 GB    | In-memory rate limiter and write buffer would suffer under multi-worker Unicorn without shared state. | Deploy Redis for shared rate limiting and session cache; migrate meta.db to hosted PostgreSQL.                |
| Enterprise Production | 10,000 - 100,000+ users | 100 GB - 10+ TB | Single-node ASGI engine would bottleneck on synchronous DB execution and external Gemini rate limits. | Horizontal autoscaling on AWS ECS/EKS; read-replica DB pools; async Celery query workers; local vLLM cluster. |

## SECTION 15 — ARCHITECTURAL DESIGN DECISIONS & TRADE-OFFS

Every major component selection involved weighing engineering trade-offs between development velocity, runtime latency, memory footprint, and defensive security.

### 1. FastAPI vs. Flask / Django

- Decision:** FastAPI with Starlette and Unicorn.
- Why:** Native async ASGI support, high request throughput, automated OpenAPI Swagger documentation, and first-class Pydantic v2 data validation.
- Alternative Considered:** Flask (synchronous by default, lacks built-in request validation), Django (excessive monolith overhead for a decoupled REST API).
- Trade-off:** FastAPI requires strict type hinting, but completely eliminates runtime type coercion bugs.

### 2. Universal Adapter Architecture vs. Direct Raw SQL

- Decision:** Abstract BaseDatabaseAdapter with concrete SQLiteAdapter and PostgreSQLAdapter.
- Why:** Decouples the query execution engine from underlying database engines. The core AI pipeline does not need to know whether the target is a local file or remote PostgreSQL database.
- Alternative Considered:** Raw SQLite scripts scattered through endpoint handlers.
- Trade-off:** Requires maintaining an abstraction layer, but allows plug-and-play addition of MySQL, Snowflake, or DuckDB adapters without touching query generation logic.

### 3. AST Parsing (sqlglot) vs. Regular Expressions

- Decision:** Pure-Python AST parsing with sqlglot.
- Why:** Regular expressions cannot handle nested SQL queries, quoted semicolons, or complex statement semantics. AST parsing provides mathematical certainty over statement types and clauses.
- Alternative Considered:** Regex pattern matching (e.g. `re.search(r'\bDROP\b', sql)`).
- Trade-off:** Adds ~2ms parsing overhead per query, but provides ironclad defense against SQL injection.

4. Two-Phase Staged Write Confirmation vs. Immediate Execution

**Decision:** Staged in-memory buffer (`_PENDING_WRITES`) requiring explicit `POST /api/confirm-write`.

**Why:** AI is probabilistic. Directly executing AI-generated write mutations against a production database creates unacceptable risk of catastrophic data loss. The confirmation modal puts a human in the loop.

**Alternative Considered:** Direct execution with a post-hoc undo log.

**Trade-off:** Requires two round-trips for write mutations, but guarantees 100% human oversight before data modifications occur.

SECTION 16 — MODULE DEPENDENCY TOPOLOGY & BLAST RADIUS

The system adheres to a strict layered dependency structure. Lower layers never depend on higher layers, preventing circular dependencies and enabling isolated unit testing.



Figure: Layered Architecture Dependency Map

16.1 Blast Radius Analysis

- **Central Engine Module:** `app/services/sql_generator.py`. If modified or corrupted, all natural language translation ceases. However, AST validation and direct DB querying remain functional.
- **Central Security Module:** `app/services/sql_validator.py`. If removed, query execution fails closed (safe by default).
- **Independent Modules:** `app/services/rag_service.py` and `app/database/sqlite_adapter.py` can be tested and modified in total isolation from web routers or LLM models.

SECTION 17 — IMPORTANT TECHNICAL COMPONENTS (JURY MUST KNOW)

Before entering the technical defense room, master the exact technical definitions, codebase locations, and architectural importance of the following twelve core components.

| Component               | Technical Definition                                                        | Code Location                    | Architectural Importance                                          | Anticipated Jury Question                                       |
|-------------------------|-----------------------------------------------------------------------------|----------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|
| AST Validator           | sqlglot Abstract Syntax Tree parser checking syntax and statement nodes.    | app/services/sql_validator.py    | Prevents SQL injection and blocks destructive DDL.                | 'Why not just use regular expressions to block DROP TABLE?'     |
| Schema-Aware RAG        | Dense semantic vector retrieval using SentenceTransformers + FAISS.         | app/services/rag_service.py      | Prevents token bloat on large schemas (> 4 tables).               | 'How does the vector index know which tables to retrieve?'      |
| Two-Tier Rate Limiter   | In-memory sliding window rate limiter: 20 req/min session, 100 global.      | app/routers/query.py             | Shields backend and Gemini API from denial-of-service hammering.  | 'How do you prevent memory leaks from inactive sessions?'       |
| Two-Phase Write Staging | In-memory staging buffer (_PENDING_WRITES) requiring explicit confirmation. | app/routers/query.py             | Eliminates risk of accidental database mutations.                 | 'What stops an attacker from executing another user's write?'   |
| Self-Correction Loop    | Iterative feedback loop (up to 3 retries) feeding errors back to LLM.       | app/routers/query.py             | Heals syntax anomalies and column mismatches autonomously.        | 'How does self-correction prevent infinite retry loops?'        |
| Universal DB Adapter    | Object-oriented abstraction pattern (BaseDatabaseAdapter).                  | app/database/base.py             | Allows uniform querying across SQLite and PostgreSQL.             | 'Can this system query remote enterprise databases?'            |
| Zero-Result Reasoning   | Evaluates zero-row SELECT queries against schema sample values.             | app/services/sql_generator.py    | Explains *why* no data matched rather than showing a blank table. | 'How do you tell the difference between a bug and empty data?'  |
| Unicode Language Gate   | Unicode block scanner (\u0B80-\u0BFF) & Thanglish lexicon matcher.          | app/services/sql_generator.py    | Enforces native Tamil and Thanglish multilingual translation.     | 'How do you detect Tamil if the user writes in English script?' |
| Duplicate INSERT Check  | AST inspection + parameterized SELECT 1 pre-check before inserting.         | app/services/execution_engine.py | Prevents duplicate records in tables lacking unique constraints.  | 'How do you prevent duplicate data from multiple clicks?'       |
| Path Masking Regex      | Regular expression stripping filesystem paths from error messages.          | app/services/execution_engine.py | Prevents local filesystem path disclosure.                        | 'Do error messages expose internal server file structures?'     |
| SQLite WAL Pragma       | Write-Ahead Logging mode (PRAGMA journal_mode=WAL).                         | app/models/meta_db.py            | Enables concurrent readers without blocking writes.               | 'Doesn't SQLite lock the entire database during writes?'        |
| Deterministic Cache     | SHA-256 hashed response cache with 10-min TTL and write invalidation.       | app/services/sql_generator.py    | Achieves sub-millisecond responses on repeated queries.           | 'How do you ensure cached results don't show stale data?'       |

SECTION 18 — TECHNICAL JURY QUESTION BANK (CATEGORIES A THROUGH R)

This section provides comprehensive, code-referenced answers to eighteen critical jury questions spanning all software engineering, AI/ML, security, and database disciplines.

**QA: Explain the high-level architecture of your project and why it is designed as a decoupled SPA.**

**Verbal Jury Defense:** “Our system uses a decoupled three-tier architecture: a React Single Page Application frontend communicating over RESTful HTTP with a Python FastAPI backend, which interfaces through universal database adapters to local or remote relational engines.”

**Technical In-Depth Explanation:** A decoupled architecture cleanly isolates UI rendering and Web Speech audio capture from heavy AI orchestration and database transactions. The FastAPI backend serves as a stateless ASGI application gateway, enforcing security headers, two-tier sliding window rate limiting, and AST parsing before any database interaction occurs. This enables independent scaling, unit testing of backend services without UI overhead, and future migration to mobile or CLI clients.

Code & File Reference: [ARCHITECTURE.md: High-Level Overview](#) | [app/main.py](#)

#### QB: How do you split multi-statement SQL strings and ensure they execute safely?

**Verbal Jury Defense:** “We use `sqlglot.parse(sql, read='sqlite')` to parse statements into distinct AST trees, ensuring semicolons within quoted string literals are never mistaken for statement delimiters.”

**Technical In-Depth Explanation:** In `app/services/execution_engine.py`, `split_sql_statements()` leverages `sqlglot` to parse the SQL text. Each statement in the resulting AST is individually transpiled and executed sequentially within a single transactional context (with `conn`). If any statement raises an exception, the entire transaction rolls back atomically.

Code & File Reference: [app/services/execution\\_engine.py: split\\_sql\\_statements\(\)](#)

#### QC: Why did you choose FastAPI over Flask or Django?

**Verbal Jury Defense:** “FastAPI delivers native asynchronous ASGI performance, automatic OpenAPI documentation, and strict runtime type validation through Pydantic v2 with zero boilerplate.”

**Technical In-Depth Explanation:** Flask is synchronous by default and requires external extensions for request schema validation and documentation. Django is a heavyweight monolithic framework with an ORM tightly coupled to server-rendered templates. FastAPI provides lightweight, high-performance ASGI throughput with native async support, ideal for handling concurrent LLM streaming and database I/O.

Code & File Reference: [app/main.py](#) | [app/routers/query.py](#)

#### QD: How does the frontend handle real-time chart rendering and visualization recommendations?

**Verbal Jury Defense:** “Our `ChartPanel` component dynamically evaluates the response payload's `chart_type` and data shape, instantiating `Chart.js` bar charts, line graphs, or KPI cards via `HTML5 Canvas`.”

**Technical In-Depth Explanation:** When the backend returns query results, the `chart_type` field specifies the visualization heuristic ('bar', 'line', 'kpi', 'table'). In `frontend/src/components/ChartPanel.jsx`, the component inspects column datatypes and row cardinality. If numeric values represent sequential time series, it renders a Line chart; if categorical groupings, a Bar chart; if a single scalar, a KPI metric card.

Code & File Reference: [frontend/src/components/ChartPanel.jsx](#) | [MessageBubble.jsx](#)

#### QE: How is the `/api/query` endpoint secured against abusive request volume?

**Verbal Jury Defense:** “We implement an in-memory two-tier sliding window rate limiter enforcing 20 requests per minute per session, and 100 requests per minute globally across all sessions.”

**Technical In-Depth Explanation:** In `app/routers/query.py`, `check_rate_limit()` maintains timestamp queues. It first checks the global key '`__global__`' against a 100 req/60s ceiling to prevent distributed quota depletion. Next, it validates the individual `session_id` against a 20 req/60s ceiling. Any violation immediately raises an HTTP 429 exception with an explicit `Retry-After` header. Crucially, expired timestamps are actively deleted on each call to prevent dictionary memory leaks.

Code & File Reference: [app/routers/query.py: check\\_rate\\_limit\(\)](#)

#### QF: How does your system prevent SQLite concurrency locking issues during high-volume reads?

**Verbal Jury Defense:** “We configure SQLite in Write-Ahead Logging mode via `PRAGMA journal_mode=WAL` and `PRAGMA synchronous=NORMAL` on engine connection.”

**Technical In-Depth Explanation:** Default SQLite uses rollback journals that acquire an exclusive lock during writes, blocking all concurrent read queries. By executing `PRAGMA journal_mode=WAL` in `app/models/meta_db.py` via SQLAlchemy connection events, write operations append to a separate WAL file, allowing concurrent readers to read from the main database file without blocking or waiting.

Code & File Reference: [app/models/meta\\_db.py: set\\_sqlite\\_pragma\(\)](#)

#### QG: How does your schema-aware RAG vector search work, and when is it bypassed?

**Verbal Jury Defense:** “We embed table schemas into 384-dimensional dense vectors using `SentenceTransformers` and index them in FAISS `IndexFlatIP`; for small schemas with 4 or fewer tables, RAG is bypassed to conserve memory.”

**Technical In-Depth Explanation:** In `app/services/rag_service.py`, `build_schema_index()` generates textual table summaries containing table names, column types, and sample values. When total tables > 4, FAISS computes exact cosine similarity against the user query, returning the top-4 most relevant tables. For schemas <= 4 tables, RAG is bypassed, returning all tables directly. This prevents over-filtering small databases and saves 150MB+ RAM on container cold-starts.

Code & File Reference: [app/services/rag\\_service.py: retrieve\\_relevant\\_tables\(\)](#)

#### QH: How do you support native Tamil script and Thanglish without specialized translation APIs?

**Verbal Jury Defense:** “We use character-level Unicode block inspection for Tamil script and lexical token matching against a colloquial Tamil dictionary for Thanglish, passing language instructions directly into the LLM system prompt.”

**Technical In-Depth Explanation:** In `app/services/sql_generator.py`, `detect_input_language()` scans characters for Unicode range `\u00B80-\u00BFF`. If present, `detected_language` is set to 'tamil'. For Latin text, it checks token intersections against a Thanglish vocabulary set ('vayasuku', 'kaatunga', 'kudu', 'pannu'). The prompt instructs Gemini to output explanations and clarifications in pure Tamil script or Thanglish, validated by an automated translation fallback loop.

Code & File Reference: `app/services/sql_generator.py: detect_input_language()`

#### QI: How do you stop SQL injection attacks attempting comment-based or stacked-statement bypasses?

**Verbal Jury Defense:** “We strip quoted string literals first, reject any query containing comment syntax (`--` or `/* */`), and parse the remainder with `sqlglot` to reject multi-statement payloads.”

**Technical In-Depth Explanation:** Attackers often use SQL comments (e.g. `SELECT * FROM users -- ; DROP TABLE...`) to truncate safety checks. In `app/services/sql_validator.py`, `_strip_strings()` replaces string literals with empty quotes so valid apostrophes are not flagged. If `'--'` or `/* */` appears in the remaining code, `validate_sql()` halts with `'injection_detected'`. Furthermore, `sqlglot` parses the query into statement trees; if more than one statement exists, it is rejected.

Code & File Reference: `app/services/sql_validator.py: validate_sql()`

#### QJ: What is your query caching strategy and how do you ensure cache consistency across data modifications?

**Verbal Jury Defense:** “We implement an in-memory response cache keyed by a deterministic SHA-256 hash of the schema signature, question, language, and context, invalidated immediately upon any confirmed write mutation.”

**Technical In-Depth Explanation:** In `app/services/sql_generator.py`, `_generate_cache_key()` computes a 16-character SHA-256 hash incorporating the schema structure and normalized question. Cached responses have a 10-minute TTL. Crucially, whenever `confirm_write()` executes an `INSERT`, `UPDATE`, or `DELETE` in `app/routers/query.py`, `clear_query_cache()` is immediately triggered, guaranteeing that subsequent `SELECT` queries never return stale data.

Code & File Reference: `app/services/sql_generator.py: _QUERY_CACHE` | `app/routers/query.py: confirm_write()`

#### QK: What is your architectural bottleneck if user traffic scales to 10,000 concurrent users?

**Verbal Jury Defense:** “Our primary bottlenecks would be in-memory state storage (rate limiter, session cache, write buffer) and synchronous database driver execution on a single ASGI node.”

**Technical In-Depth Explanation:** To scale to 10,000 users, we would migrate in-memory state (`_QUERY_TIMESTAMPS`, `_QUERY_CACHE`, `_PENDING_WRITES`) to a distributed Redis cluster, migrate synchronous SQLite adapter calls to asynchronous drivers (`aiosqlite` and `asyncpg`), deploy Uvicorn across an autoscaling container cluster (AWS ECS/EKS), and point read queries to PostgreSQL read replicas.

Code & File Reference: `LIMITATIONS.md: Section 1.2 & Section 2`

#### QL: Explain how your Dockerfile optimizes image size and runtime RAM usage.

**Verbal Jury Defense:** “We use a multi-stage Docker build separating Node asset compilation from a Python slim runtime, installing CPU-only PyTorch to avoid 3.5GB of CUDA bloat.”

**Technical In-Depth Explanation:** Stage 1 compiles React assets with Node 20. Stage 2 copies the pre-built dist folder into `python:3.11-slim`. It installs PyTorch using `--index-url https://download.pytorch.org/whl/cpu`, reducing the image size from 4.5GB to ~650MB. Runtime flags like `OMP_NUM_THREADS=1` clamp background thread pools, keeping the container comfortably inside Render's 512MB RAM free-tier limit.

Code & File Reference: `Dockerfile: Stages 1 and 2`

#### QM: How does the autonomous self-correction loop heal a query that fails with an unknown column error?

**Verbal Jury Defense:** “The execution engine catches `sqlite3.OperationalError`, extracts the database error string, and passes the failed SQL along with the error message back to Gemini in `regenerate_sql()` for automated correction.”

**Technical In-Depth Explanation:** In `app/routers/query.py`, the `while attempt <= MAX_RETRIES` loop intercepts database execution errors in Step B. If the database returns `'no such column: pat_name'`, `regenerate_sql()` injects this exact error into a targeted correction prompt. Gemini synthesizes corrected SQL referencing the valid column `'name'`, which is re-validated and re-executed. The turn is logged in `query_history` with `self_corrected=1`.

Code & File Reference: `app/routers/query.py: lines 617-658` | `app/services/sql_generator.py: regenerate_sql()`

#### QN: Why do you stage write mutations in memory rather than executing them inside a temporary staging table?

**Verbal Jury Defense:** “In-memory staging in `_PENDING_WRITES` requires zero database schema alterations, zero temporary table cleanup overhead, and completely isolates unconfirmed mutations from the database engine.”

**Technical In-Depth Explanation:** Creating temporary staging tables requires executing DDL on the connected database, which alters schema state, creates race conditions among concurrent sessions, and risks orphaned tables on disconnect. Staging the validated SQL text and explanation in `_PENDING_WRITES` with a short expiration lifecycle guarantees that the database engine remains untouched until the user explicitly confirms.

Code & File Reference: `app/routers/query.py: _PENDING_WRITES`

#### QO: What are the honest technical limitations of this project?

**Verbal Jury Defense:** “Our system requires Chromium-based browsers for Web Speech API transcription, relies on single-node in-memory state for rate limiting and write buffers, and currently supports PostgreSQL via synchronous SQLAlchemy drivers rather than native async drivers.”

**Technical In-Depth Explanation:** In `LIMITATIONS.md`, we document that non-Chromium browsers must fall back to text input, server restarts clear unconfirmed staged writes, and high-concurrency production requires an external Redis state store. We adhere strictly to engineering honesty rather than claiming unverified capabilities.

Code & File Reference: `LIMITATIONS.md`: Section 1

#### QP: Why did you use Gemini Flash-Lite instead of OpenAI GPT-4o or local Ollama?

**Verbal Jury Defense:** “Gemini Flash-Lite offers superior token latency (<1s response), high rate limits on developer tiers, native Tamil multilingual fluency, and zero host GPU memory consumption.”

**Technical In-Depth Explanation:** GPT-4o introduces higher API latency and cost per token. Hosting a local Ollama LLM (e.g. CodeLlama 13B) requires at least 8GB of Dedicated GPU VRAM, which is impossible on cost-effective cloud containers like Render 512MB RAM. Gemini Flash-Lite provides the ideal balance of speed, cost, and multilingual accuracy.

Code & File Reference: `app/services/sql_generator.py`: `MODELS_TO_TRY`

#### QQ: What prevents a malicious user from approving a staged write that belongs to another active user's session?

**Verbal Jury Defense:** “Our `confirm_write` endpoint strictly verifies that `payload.session_id` matches the `session_id` stored in the staged `_PENDING_WRITES` dictionary, returning HTTP 403 Forbidden on any mismatch.”

**Technical In-Depth Explanation:** In `app/routers/query.py` line 774, when `POST /api/confirm-write` is called, the handler checks if `pending.get('session_id') != payload.session_id`. If a user attempts to guess or submit another session's `query_id`, the request is blocked immediately with HTTP 403, and the attempt is logged as an unauthorized security event.

Code & File Reference: `app/routers/query.py`: lines 772-785

#### QR: Show me exactly where you prevent duplicate records from being inserted into the database.

**Verbal Jury Defense:** “In `app/services/execution_engine.py`, `check_duplicate_insert()` parses the INSERT AST, extracts columns and values, and executes a parameterized `SELECT 1` pre-check before inserting.”

**Technical In-Depth Explanation:** Lines 94-152 of `execution_engine.py` inspect the incoming statement using `sqlglot.exp.Insert`. It extracts target table and column names, builds a parameterized query `'SELECT 1 FROM table WHERE col1=? AND col2=? LIMIT 1'`, and executes it with the literal values. If a matching row already exists, execution is skipped and a notice is returned, preventing duplicate records even if the table lacks a `UNIQUE` constraint.

Code & File Reference: `app/services/execution_engine.py`: `check_duplicate_insert()`

## SECTION 19 — ADVERSARIAL HARD JURY CROSS-QUESTIONS

These cross-questions simulate aggressive challenges from cynical software architects and AI reviewers. Each scenario provides an unassailable engineering defense grounded in the actual codebase.

### Cross-Question 1: 'LLMs hallucinate. Why should any enterprise trust your SQL queries?'

**Verbal Rebuttal:** “We never trust the LLM's raw output. The LLM acts solely as a semantic proposal engine; every query must pass three deterministic gates—Schema Grounding, AST Security Validation, and Transactional Execution—before reaching data.”

#### Deep Engineering Defense:

- Schema Grounding:** The LLM prompt is dynamically populated with verified table names, column data types, and distinct sample values extracted directly via `PRAGMA table_info` and `information_schema`. The LLM is never allowed to guess column names.
- Boundary Checking:** Before synthesis, our ambiguity and data-availability gates reject out-of-schema queries (e.g. asking for airline flights on a hospital database) with `data_available: false`.
- AST Validation:** `sqlglot` parses the proposed query into an Abstract Syntax Tree, mathematically verifying that statement types match the session mode, rejecting destructive DDL, and verifying `WHERE` clauses.
- Autonomous Self-Correction:** If a syntax or column error occurs, our retry loop feeds the exact SQLite operational error back to Gemini to repair the query automatically.

### Cross-Question 2: 'What happens if a user asks a query in Thanglish that confuses Tamil and English words?'

**Verbal Rebuttal:** “Our system treats Thanglish as a first-class citizen using a dual-mode token filter and conversational confirmation flow.”

**Deep Engineering Defense:** In `app/services/sql_generator.py`, `detect_input_language()` checks token sets against a compiled lexicon of colloquial Tamil particles (`kaatunga`, `vayasuku`, `kudunga`, `irukku`). When detected, the prompt instructs Gemini to translate the intent into standard SQL while returning an explanation acknowledging the colloquial phrasing. If a name is phonetically ambiguous, `find_near_miss_value()` intercepts the query and asks: *'Did you mean Dr. Priya Patel?'* before executing.

### Cross-Question 3: 'What happens if the backend crashes while a write mutation is staged?'

**Verbal Rebuttal:** "The system fails safely. Because mutations are staged in memory and only executed upon explicit confirmation, a crash results in zero uncommitted data mutations."

**Deep Engineering Defense:** Staged write statements reside in the in-memory `_PENDING_WRITES` dictionary. If the Unicorn server restarts, the dictionary clears. The database files remain completely intact. When the user attempts to confirm, the backend checks database history; if unexecuted, it prompts the user to re-submit the query. At no point is a half-written transaction left lingering.

## SECTION 20 — 'SHOW ME THE CODE' CODEBASE INSPECTION WALKTHROUGHS

Be prepared for jury members who interrupt slides and demand: *'Open your editor and show me the exact line where this occurs.'*

### 1. 'Show me where you strip SQL comments to prevent injection bypasses.'

- **File:** `nl2sql-backend/app/services/sql_validator.py`
- **Lines:** Lines 17-30 (`_strip_strings()` and `_strip_comments_and_strings()`) and Lines 59-66.
- **Exact Code:** `without_strings = _strip_strings(sql_string); if '--' in without_strings or '/' in without_strings: return {'valid': False, 'reason': 'injection_detected'}`

### 2. 'Show me where you enforce that UPDATE and DELETE must have a WHERE clause.'

- **File:** `nl2sql-backend/app/services/sql_validator.py`
- **Lines:** Lines 126-141.
- **Exact Code:** `if isinstance(stmt, (exp.Update, exp.Delete)): if not stmt.find(exp.Where): return {'valid': False, 'reason': 'missing_where_clause', 'message': 'UPDATE/DELETE queries must include a WHERE clause for safety.'}`

### 3. 'Show me where you verify session ownership during write confirmations.'

- **File:** `nl2sql-backend/app/routers/query.py`
- **Lines:** Lines 773-784 and Lines 793-803.
- **Exact Code:** `if pending.get('session_id') != payload.session_id: raise HTTPException(status_code=403, detail='Forbidden: This pending write query belongs to a different session.')`

### 4. 'Show me where you sanitize server file paths from database error messages.'

- **File:** `nl2sql-backend/app/services/execution_engine.py` & `app/database/sqlite_adapter.py`
- **Lines:** Lines 18-21 (regex definition) and Lines 80-85 of `execution_engine.py`.
- **Exact Code:** `_FILE_PATH_RE = re.compile(r'(?[a-zA-Z]:[\\V][^\\s:"\';,;])+/(?:home|app|tmp|var|usr|etc|root|data)/[^\s:"\';,;]+/[w./\]+\.(?:db|sqlite|sqlite3)\b)', re.IGNORECASE);  
err_msg = _FILE_PATH_RE.sub('[database]', err_msg)`

## SECTION 21 — VERIFICATION & TESTING MATRIX

The NL2SQL backend undergoes rigorous automated quality assurance via pytest, pytest-cov, and Starlette's TestClient. The entire suite consists of **166 automated test cases** across 17 test modules, achieving a **100% pass rate**.

| Test Module                  | Target Subsystem       | Test Classes / Focus Areas                                       | Tests Count | Pass Rate |
|------------------------------|------------------------|------------------------------------------------------------------|-------------|-----------|
| test_sql_validator.py        | AST Security Gate      | Valid SELECTs, comment stripping, DDL blocks, WHERE clauses      | 44 tests    | 100% PASS |
| test_security.py             | API Gateway & Security | Security headers, file upload guards, two-tier rate limiting     | 23 tests    | 100% PASS |
| test_conversations.py        | Thread Persistence     | CRUD operations, Turn isolation, Follow-up context queries       | 16 tests    | 100% PASS |
| test_adapters_and_writes.py  | DB Adapters & Writes   | SQLite/Postgres adapters, schema extraction, confirm-write       | 7 tests     | 100% PASS |
| test_execution_engine.py     | Execution Engine       | Safe SELECT execution, path leak masking, duplicate checks       | 11 tests    | 100% PASS |
| test_rag.py                  | FAISS RAG Service      | Small schema bypass, large schema retrieval, embedding lifecycle | 11 tests    | 100% PASS |
| test_clarification.py        | Ambiguity Engine       | Underspecified queries, unconstrained ranking terms ('top')      | 10 tests    | 100% PASS |
| test_multilingual.py         | Multilingual NLP       | Tamil script Unicode, Thanglish vocabulary, translation fallback | 9 tests     | 100% PASS |
| test_voice_correction.py     | Phonetic Correction    | Levenshtein distance, near-miss speech correction                | 9 tests     | 100% PASS |
| test_data_value_integrity.py | Data Availability      | Out-of-schema entity detection, schema boundaries                | 8 tests     | 100% PASS |
| test_self_correction.py      | Self-Correction Loop   | Iterative syntax repair, execution error healing (3 attempts)    | 8 tests     | 100% PASS |
| test_connection.py           | DB Connection          | Demo hospital/ecommerce connect, schema introspection            | 7 tests     | 100% PASS |
| test_health.py               | Liveness Probe         | HTTP /health status, content-type verification                   | 3 tests     | 100% PASS |

## SECTION 22 — DEPLOYMENT, CONTAINERIZATION & DEVOPS PIPELINE

NL2SQL implements a unified deployment model where pre-built React frontend assets are served directly by FastAPI from its static root, eliminating cross-origin CORS overhead in production.

### 22.1 Multi-Stage Dockerfile Architecture

- **Stage 1 (Frontend Builder):** Uses node:20-alpine. Installs dependencies via npm ci and compiles production assets with npm run build into /frontend/dist. The API base URL defaults to same-origin (VITE\_API\_BASE\_URL="").
- **Stage 2 (Runtime Container):** Uses python:3.11-slim. Installs CPU-only PyTorch via --index-url https://download.pytorch.org/whl/cpu, reducing image size by ~3.5GB. Copies backend application code and compiled frontend assets.
- **Thread Clamping:** Sets OMP\_NUM\_THREADS=1, MKL\_NUM\_THREADS=1, and TORCH\_NUM\_THREADS=1 to prevent multithreaded math libraries from blowing through RAM limits.
- **Healthcheck & Cold Start:** Defines a Docker HEALTHCHECK polling http://localhost:8000/health with a start-period=60s grace window, preventing premature container terminations on slow cloud cold-starts.

## SECTION 23 — TECHNICAL WEAKNESSES, RISKS & PRODUCTION REMEDIATION

In accordance with rigorous software engineering ethics, this audit outlines the system's operational boundaries and provides concrete remediation strategies for enterprise scale.

| Identified Weakness / Risk | Root Cause in Current Implementation                                  | Operational Impact                                                                           | Production Remediation Strategy                                                                                                     |
|----------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| In-Memory Rate Limiting    | Timestamps stored in local dict ( <code>_QUERY_TIMESTAMPS</code> )    | Rate limit counts reset on restart; does not scale across multiple Uvicorn worker processes. | Replace in-memory dictionary with a distributed Redis sliding-window counter using Redis EVAL scripts.                              |
| In-Memory Write Buffer     | Staged mutations stored in <code>_PENDING_WRITES</code> buffer        | Unconfirmed write operations are lost if server restarts before user clicks confirm.         | Persist staged writes to <code>meta.db</code> in a 'pending_writes' table with a 5-minute expiration timestamp.                     |
| Web Speech API Dependency  | Relies on browser-native <code>SpeechRecognition</code> interface     | Voice transcription is limited to Chromium browsers (Chrome, Edge); unavailable in Firefox.  | Integrate an optional server-side Whisper API audio upload endpoint for universal browser support.                                  |
| Synchronous DB Drivers     | <code>sqlite3</code> and <code>psycopg2</code> use blocking I/O calls | Long-running queries can block the single-worker ASGI event loop temporarily.                | Migrate adapter implementations to asynchronous drivers: <code>aiosqlite</code> for SQLite and <code>asyncpg</code> for PostgreSQL. |
| PostgreSQL Driver Bundling | <code>psycopg2-binary</code> is optional to conserve image size       | Connecting to live PostgreSQL requires ensuring driver is installed in Python environment.   | Package <code>asyncpg</code> in <code>core requirements.txt</code> once container memory limits permit expanded baseline footprint. |

## SECTION 24 — 5-MINUTE TECHNICAL PRESENTATION DELIVERY SCRIPT

Follow this exact timed structure during your five-minute technical presentation before the jury:

- 0:00 - 0:30 — The Problem:** 'Good morning, esteemed jury. Today, relational databases power enterprise operations, yet 80% of business stakeholders cannot extract insights without burdening data engineering teams. Furthermore, naive LLMs hallucinate schema details, risk SQL injection, and lack regional language support.'
- 0:30 - 1:00 — The Solution:** 'We built NL2SQL: an enterprise conversational query engine supporting English, native Tamil script, and Thanglish. It guarantees mathematical query safety through AST validation, stages write mutations for human confirmation, and dynamically renders Chart.js visualizations.'
- 1:00 - 2:00 — Architecture & Innovation:** 'Our architecture decouples a React Vite frontend from a FastAPI ASGI backend. Queries pass through six defensive gates: rate limiting, language/phonetic normalization, schema-aware FAISS vector RAG, Gemini Flash-Lite synthesis, sqlglot AST parsing, and an autonomous self-correction loop that heals syntax errors up to three times.'
- 2:00 - 3:30 — Deep Technical Implementation:** 'Notice our universal database abstraction: `BaseDatabaseAdapter` allows live SQLite, PostgreSQL, or CSV/SQL uploads to be queried identically. For write safety, `SELECT` queries run immediately, while `INSERT`, `UPDATE`, and `DELETE` operations are staged in memory with session ownership authorization. Transactions commit atomically or roll back completely.'
- 3:30 - 4:30 — Live Demonstration:** [Demonstrate connecting hospital DB → ask in Thanglish 'Cardiology doctor yaaru' → inspect generated SQL → show `ChartPanel` visualization → demonstrate typing write query & `ConfirmModal`.]
- 4:30 - 5:00 — Verification & Roadmap:** 'Our codebase is proven with 166 automated test cases at 100% pass rate. It is containerized via multi-stage Docker and optimized for 512MB RAM cloud hosting. We are ready for your technical questions.'

## SECTION 25 — 30-SECOND TECHNICAL ELEVATOR PITCH

### VERBAL DEFENSE: 30-SECOND ELEVATOR PITCH

"NL2SQL is an enterprise conversational AI query system that translates natural language—in English, Tamil, and Thanglish—into verified, production-safe SQL across SQLite and PostgreSQL databases. Unlike naive LLM wrappers, we enforce a strict defense-in-depth pipeline: schema-aware FAISS vector RAG, mathematical AST validation via sqlglot, two-phase staged write confirmations, and an autonomous self-correction retry loop. Verified with 166 automated test cases, the system delivers sub-second deterministic responses, zero-row result reasoning, and automated Chart.js visualizations."

## SECTION 26 — 2-MINUTE ARCHITECTURAL EXECUTIVE BRIEFING

When granted two minutes to provide a deeper technical summary, deliver the following structured brief:

- **System Topology:** 'Our system is architected as an asynchronous, decoupled client-server application. The client is a React 18 SPA built with Vite, TailwindCSS, Chart.js, and browser-native Web Speech API. The backend is powered by Python 3.11 and FastAPI over Uvicorn, implementing security headers, sliding-window rate limiting, and session-isolated database adapters.'
- **AI Pipeline & Schema Grounding:** 'We use Google Gemini Flash-Lite configured at temperature 0.1 for deterministic synthesis. To handle enterprise schemas without token bloat, our schema-aware RAG embeds table definitions using SentenceTransformers (all-MiniLM-L6-v2) into a FAISS IndexFlatIP vector index, dynamically retrieving the top-4 relevant tables for schemas larger than 4 tables. Schemas under 4 tables bypass RAG to conserve RAM.'
- **Defensive Security & Validation:** 'Security is enforced through sqlglot AST parsing. We pre-strip string literals to block SQL comment bypasses, unconditionally reject destructive DDL commands, and require explicit WHERE clauses on all UPDATE and DELETE statements. Write operations are never executed directly; they are staged in memory with strict session ownership verification, requiring explicit two-phase confirmation.'
- **Resilience & Performance:** 'If an LLM syntax error occurs, our self-correction loop catches the database exception and passes the failure context back to Gemini to heal the query across up to 3 attempts. Query responses are cached with a deterministic SHA-256 signature and a 10-minute TTL, automatically invalidated upon confirmed data mutations. The entire system is proven with 166 automated pytest cases passing at 100%.'

## SECTION 27 — RAPID 10-MINUTE PRE-JURY REVISION SHEET

Review these key facts, file mappings, algorithms, and numbers immediately before entering your technical defense:

| Metric / Concept        | Exact Code Value / Configuration                            | Architectural Reference                                 |
|-------------------------|-------------------------------------------------------------|---------------------------------------------------------|
| Test Suite Results      | 166 Passed, 0 Failed, 0 Skipped (100% Pass)                 | TEST_REPORT.md   tests/conftest.py                      |
| Rate Limiting Limits    | 20 req/min per session; 100 req/min global                  | app/routers/query.py: check_rate_limit()                |
| Response Cache TTL      | 600 seconds (10 minutes) with write invalidation            | app/services/sql_generator.py: _QUERY_CACHE             |
| Self-Correction Retries | Up to 2 retries (original attempt + 2 retries = 3 attempts) | app/routers/query.py: MAX_RETRIES = 2                   |
| RAG Threshold           | Bypassed for <= 4 tables; top-4 retrieval for > 4 tables    | app/services/rag_service.py: retrieve_relevant_tables() |
| Embedding Dimension     | 384-dimensional dense vectors (all-MiniLM-L6-v2)            | app/services/rag_service.py: IndexFlatIP                |
| Gemini Model            | gemini-3.1-flash-lite at temperature = 0.1                  | app/services/sql_generator.py: MODELS_TO_TRY            |
| AST Parser              | sqlglot (pure Python AST parsing & transpilation)           | app/services/sql_validator.py: validate_sql()           |
| SQLite Pragma Mode      | PRAGMA journal_mode=WAL; PRAGMA synchronous=NORMAL          | app/models/meta_db.py: set_sqlite_pragma()              |
| Write Buffer Key        | _PENDING_WRITES dictionary keyed by query_id                | app/routers/query.py: _PENDING_WRITES                   |
| File Upload Ceiling     | 5MB maximum; whitelisted to .csv and .sql extensions        | app/routers/connect_db.py: upload_data()                |
| Tamil Unicode Range     | \u0B80 through \u0BFF character block                       | app/services/sql_generator.py: detect_input_language()  |

## SECTION 28 — EXHAUSTIVE TECHNICAL GLOSSARY

Authoritative definitions for all advanced software engineering and AI terms utilized in the codebase:

- **Abstract Syntax Tree (AST):** A tree representation of the abstract syntactic structure of source code or SQL statements, where each node denotes a construct (e.g. exp.Select, exp.Where).
- **Retrieval-Augmented Generation (RAG):** An AI architecture that retrieves external knowledge (such as database schemas) using vector similarity search to ground LLM generation and eliminate hallucinations.
- **Cosine Similarity:** A mathematical measure of similarity between two non-zero vectors calculating the cosine of the angle between them, evaluated as their normalized dot product.

- **FAISS (Facebook AI Similarity Search):** A high-performance library for efficient similarity search and dense vector clustering.
- **Write-Ahead Logging (WAL):** A database transaction logging technique where modifications are written to a separate WAL file before being applied to the database, enabling concurrent reads during writes.
- **Two-Phase Confirmation:** A safety design pattern where destructive operations are first parsed, validated, and staged in memory, requiring an explicit second affirmative request before execution.
- **Sliding Window Counter:** A rate limiting algorithm tracking individual request timestamps within a moving time window (60s) to enforce smooth, burst-resistant request limits.
- **Content Security Policy (CSP):** An HTTP response header restricting the sources from which scripts, styles, fonts, and frames may be loaded by the browser, mitigating XSS and clickjacking.
- **Deterministic Caching:** Hashing query inputs and schema state via cryptographic SHA-256 to ensure identical inputs always map to identical cache slots without non-deterministic key drift.